

An Improved DSATUR-Based Branch-and-Bound Algorithm for the Vertex Coloring Problem

Fabio Furini

Université Paris-Dauphine, PSL Research University, CNRS, LAMSADE, 75016 Paris, France

Virginie Gabrel

Université Paris-Dauphine, PSL Research University, CNRS, LAMSADE, 75016 Paris, France

Ian-Christopher Ternier

Université Paris-Dauphine, PSL Research University, CNRS, LAMSADE, 75016 Paris, France

Given an undirected graph, the Vertex Coloring Problem (VCP) consists of assigning a color to each vertex of the graph in such a way that two adjacent vertices do not share the same color and the total number of colors is minimized. DSATUR-based Branch-and-Bound algorithm (DSATUR) is an effective exact algorithm for the VCP. One of its main drawback is that a lower bound is computed only once and it is never updated. We introduce a reduced graph which allows the computation of lower bounds at nodes of the branching tree. We compare the effectiveness of different classical VCP bounds, plus a new lower bound based on the 1-to-1 mapping between VCPs and Stable Set Problems. Our new DSATUR outperforms the state of the art for random VCP instances with high density, significantly increasing the size of instances solved to proven optimality. © 2016 Wiley Periodicals, Inc. NETWORKS, Vol. 69(1), 124–141 2017

Keywords: graph coloring; DSATUR; branch-and-bound algorithm; bounding technique; computational experiments; exact algorithm

1. INTRODUCTION

Given an undirected graph $G = (V, E)$ with $|V| = n$ vertices and $|E| = m$ edges, a *stable set* is a subset $V_S \subseteq V$ of non-adjacent vertices, i.e., $\forall u, v \in V_S, (u, v) \notin E$. A *coloring* C of G is a partition of V into k non-empty *stable sets*: $C = \{V_1, \dots, V_k\}$, where all vertices belonging to V_i are colored with the same color i ($i = 1, \dots, k$). The *chromatic number* of G , denoted by $\chi(G)$, is the minimum number of stable sets (or equivalently colors) in a coloring of G and the *Vertex*

Coloring Problem (VCP) is the problem of determining the chromatic number of the graph G .

The VCP is one of the classical NP-hard problems (see Garey and Johnson [13]) in graph theory with application in many areas including: scheduling, timetabling, register allocation, frequency assignment, communication networks, and many others (see [1, 2, 4, 5, 9, 12, 18, 24, 31]). We address the interested reader to Malaguti and Toth [20] for a complete survey on the topic.

The VCP has received a large amount of attention in the last decades and many articles investigated an exact implicit enumeration algorithm called *DSATUR-based Branch-and-Bound algorithm* (DSATUR), first introduced by Brélaz [3] and then improved by Sewell [30] and San Segundo [27]. It is a Branch-and-Bound algorithm where, at each node, the children nodes are created by assigning feasible colors to a uncolored vertex; thus at each node of the branching tree, we have a partial coloring of G and at each leaf we have a coloring of G . Formally, a *partial coloring* $\tilde{C}$ of G is a partition of a subset of vertices $\tilde{V} \subset V$ into $\tilde{k}$ non-empty stable sets or colors ($\tilde{C} = \{\tilde{V}_1, \dots, \tilde{V}_{\tilde{k}}\}$), while the remaining vertices $V \setminus \tilde{V}$ are uncolored. Many rules have been proposed in the literature to determine the sequence of vertices to color (see Section 2 for further details on DSATUR). It is worth mentioning that DSATUR has also been successfully applied to other variants of VCPs, see, for example, Méndez-Díaz et al. [22].

In all the DSATUR versions proposed in the literature, a lower bound is computed once at the root node of the algorithm as a heuristic maximal clique and it is never updated. A second trivial lower bound also used in the literature is the number of colors $\tilde{k}$ of a partial coloring $\tilde{C}$. The principal idea of this manuscript consists of updating and improving the quality of the lower bound during the branching scheme. To do that, we introduce a *Reduced Graph* associated with a

partial coloring which allows us to update the lower bounds. We implement and compare the classical lower bounds for VCP, i.e., the clique number, a bound based on the stability number, the fractional chromatic number and the Hoffman bound. Since all these bounds turn out to be useful only in reducing the number of nodes of DSATUR but not the computation time, we investigate a new bound based on a 1-to-1 mapping between VCPs and Stable Set Problems. The mapping has been originally proposed in [8], for the VCPs and the Max Coloring Problem. In [7], the transformation is extended to the Equitable Coloring Problem and the Bin Packing Problem with Conflicts. These problems are then directly solved by reformulating them as Maximum Weighted Stable Set Problems on an associated graph (plus additional constraints). To the best of our knowledge, our manuscript is the first one which proposes to use the mapping instead to derive bounds for the chromatic number. Thanks to these new bounds, we manage to reduce both the number of nodes and the computation time of DSATUR for random VCP instances with high density. A preliminary version of this manuscript appeared in [11].

For random graphs DSATUR outperforms other exact algorithms, see San Segundo [27]. For DIMACS VCP instances instead, Branch-and-Price algorithms based on the Integer Linear Programming (ILP) formulation of Mehrotra and Trick [21] guarantees the best performances. Many articles study ways of improving this class of exact algorithms. Malaguti et al. [19] focus on finding the most efficient way to solve the pricing subproblem, proposing a tabu-search meta-heuristic which speeds up the computational convergence. Gualandi and Malucelli [15] propose instead to solve the pricing subproblem using constraint programming techniques. Held et al. [6] also work on this formulation tackling numerical difficulties in the context of column generation, deriving a way of computing numerically safe bounds. Finally, Morrison et al. [23] work on new branching rules that preserve the graph structure at each node of the branching tree.

The remainder of the paper is organized as follows. In Section 2, we recall DSATUR and present different vertex selection rules. In Section 3, we present and computationally compare the VCP lower bounds. In Section 4, we introduce the Reduced Graph used to compute VCP lower bounds starting from a partial coloring. In Section 5, we discuss extensive computational results and depict further possible lines of research on the topic.

2. STATE OF THE ART: DSATUR-BASED BRANCH-AND-BOUND ALGORITHM

In this section, we recall the DSATUR-based Branch-and-Bound algorithm called for brevity DSATUR in the following. We base our review on the notation offered by San Segundo [27]. The algorithm is based on $DSATUR_h$ (see Brélaz [3]) which is a greedy heuristic algorithm where each vertex $u \in V$ is iteratively colored with a feasible color. A color is considered feasible for a vertex u if none of the vertices in its neighborhood ($N(u)$) have already taken that

color. Given a partial coloring $\tilde{C}$ and a vertex $u \in V$, the *saturation degree* $DSAT(u, \tilde{C})$ corresponds to the number of different colors in $N(u)$. At each iteration of $DSATUR_h$, the vertex with the highest DSAT value is colored until a feasible heuristic coloring of the entire graph G is obtained; the number of used colors is then a valid upper bound for $\chi(G)$.

An exact Branch-and-Bound algorithm can be derived from $DSATUR_h$. Given a partial coloring and an uncolored vertex $u \in V$, instead of fixing its color in a greedy way, a branching tree is created by coloring u with all the feasible colors already used in the partial coloring plus a new one. At each node of this branching tree, we are given a partial coloring $\tilde{C}$ with $\tilde{k}$ colors, an upper bound (UB) and a lower bound (LB) on $\chi(G)$. Trivially, $\tilde{k}$ can be used as a lower bound for $\chi_{\tilde{C}}(G)$, i.e., the chromatic number of G partially colored by $\tilde{C}$. Another LB can be found at the root node of the Branch-and-Bound algorithm by determining a clique in G . This LB is typically never updated in the classical implementation of DSATUR.

In Algorithm 1 and Algorithm 2, we give the pseudo-code of DSATUR. Precisely, Algorithm 1 receives as input the graph G to color, computes a maximal clique using a heuristic algorithm (a maximum clique if possible) to initialize LB and it produces as output the optimal coloring C^* of value $\chi(G)$. The algorithm used to compute the maximal clique is denoted by $MaxClique(G)$ in the pseudo-code of Algorithm 1. In Algorithm 2, the mechanism to create the children nodes is described, i.e., after an uncolored vertex is selected and if $\max\{\tilde{k}, LB\} < UB$ (where $\tilde{k}$ is the number of different color in $\tilde{C}$), up to $\tilde{k} + 1$ children nodes are created by coloring the selected vertex with all the feasible colors in $\tilde{C}$ plus a new one. In case all vertices are colored and if $\tilde{k} < UB$, the best incumbent solution value and the best solution are updated, respectively.

Algorithm 1 DSATUR

Data: $G = (V, E)$: graph to color
Result: optimal coloring C^* of value $\chi(G)$
1 $LB \leftarrow MaxClique(G)$, $UB \leftarrow n$;
2 $DSATUR(\emptyset)$;
3 **return** C^*

The basic Vertex Selection Rule (VSR), proposed in Brélaz [3], consists of coloring the vertex with the maximum DSAT value, thus it minimizes the number of children nodes. During the execution of DSATUR, it often happens that many different vertices share the same maximum DSAT value, i.e., creating possible ties. The following tie-breaking rules have been introduced in the literature:

- (i) In Brélaz [3], ties are broken considering the maximum degree or, in case of further ties, the lexicographical order is used. The computational complexity of this rule is $O(n^2)$.

Algorithm 2 DSATUR ($\tilde{C}$)

```
1 if all the vertices are colored then
2   if  $\tilde{k} < UB$  then
3      $C^* \leftarrow \tilde{C}, UB \leftarrow \tilde{k};$ 
4   end
5 else
6   select an uncolored vertex  $v$ ;
7   for every feasible color  $i \in \tilde{C}$  plus a new one do
8      $\hat{C} \leftarrow \tilde{C}$ , add  $v$  in  $\hat{V}_i$ ;
9     if  $\max\{\hat{k}, LB\} < UB$  then
10       DSATUR( $\hat{C}$ );
11     end
12   end
13 end
```

- (ii) In Sewell [30], ties are instead broken considering the maximum number of common available colors in the neighborhood of uncolored vertices. The computational complexity of this rule is $O(n^3)$.
- (iii) In San Segundo [27], the Sewell rule is extended considering only uncolored vertices that are also candidates in the tie. In the worst case, the computational complexity is the same as Sewell's rule but it is faster on average (see [27] for further details).

The fastest DSATUR algorithm is the one proposed by San Segundo [27]. Since its implementation is not available, we re-implement the same algorithm following the description given in [27]. In particular:

- the same preprocessing is implemented. Dominated vertices are removed (a vertex $u \in V$ is dominated if there exists a vertex $v \in V$ such that $N(u) \subseteq N(v)$). Vertices $v \in V$ such that $|N(v)| < LB$ are also removed;
- in [27] an exact algorithm (see [28]) for the maximum clique problem is used. Since determining the maximum clique can be computationally expensive, a time limit of 5 s is imposed in [27]. We solve the maximum clique problem using Cliquer [25] (since the implementation of the algorithm used in [27] is not available) and the same time limit. The vertices in this clique are colored before starting DSATUR;
- the vertex ordering is similar to the one used in [27], i.e., vertices belonging to the initial clique (LB) are ordered first in V and then the remaining vertices are sorted by non-increasing degree order ($|N(u)|, u \in V$).

In Table 1, we show that our implementation has the same computational efficiency as DSATUR [27] and it outperforms current leading exact column generation based algorithms.

We test the same benchmark of random instances introduced in [27]. Each line in Table 1 contains average values over 50 instances of a given number of vertices n and a given density $d = \frac{2 \cdot m}{n \cdot (n-1)}$. The column identified by # reports the number of instances with the same value of n and d . For $n = 75, 80$ only 49 instances are available for each value of density and only 48 for $n = 70$ and

$d = 0.7$. This entire benchmark of instances can be downloaded at <http://www.lamsade.dauphine.fr/coloring>. In Table 1, we report the tests of three exact algorithms, i.e., two different implementations of DSATUR [27] and two different Branch-and-Price algorithms. The table contains the following information imposing a common time limit of 1,200 s:

- UB : the average number of colors (the chromatic number or an upper bound in case the time limit is reached).
- LB : the average LB obtained (for MMT-BP [19], LB represents the bound obtained at the root node while, for Exactcolors, LB represents the global final bound).
- $nodes$: the average number of explored nodes (not available for DSATUR [27] and MMT-BP [19]). The averages are computed only for instances solved to proven optimality.
- $time$: the average computation time in seconds (time limits are not considered in the averages).
- $fails$: the number of instances that cannot be solved within the time limit.

The first and the second group of columns are the results of two different implementations of DSATUR [27]: the first group reports the results directly taken from [27] while the second group reports the results of our re-implementation of the algorithm. Then we report the results of two Branch-and-Price algorithms. The first one is the Branch-and-Price algorithm described in [19] (called *MMT-BP*) and the second one is the Branch-and-Price algorithm described in [6] (called *Exactcolors*). Since the implementation of MMT-BP is not available, in Table 1 we report the results taken from [27]. For Exactcolors, the code is available at <https://github.com/heldstephan/exactcolors> and we test this algorithm on our machine (using default values for all the parameters).

To compare the computation times obtained on our machine and the machine used in [27], we execute the benchmark algorithm dfmax achieving the following results: the *user times* are 0.0, 0.18, 1.1, and 4.24 for instances r200.5, r300.5, r400.5, and r500.5, respectively. Comparing our results to the results reported in [27], we conclude that our machine is 1.28 times faster than their machine. Accordingly, we scale the computation time taken from [27] in Table 1.

Table 1 shows that our reimplementation of DSATUR slightly outperforms the implementation in [27]. This is due to a careful re-implementation of all the data structures and functions of the algorithm. Our code is available at www.lamsade.dauphine.fr/coloring. As far as the comparison between DSATUR and the Branch-and-Price algorithms is concerned, we obtain the same results as in [27], i.e., DSATUR is the best algorithm for this benchmark of instances.

An important class of instances for the VCP is available in the DIMACS benchmark library. We test our reimplementation on this entire testbed obtaining, also in this case, a similar computational behavior of DSATUR [27]. For this set of instances, the Branch-and-Price algorithms remain the

TABLE 1. Comparison between exact algorithms on the benchmark of instances introduced in [27].

Instance			DSATUR							Branch-and-Price algorithms									
			[27]			Reimplementation [10]				MMT-BP [19]				Exactcolors [6]					
n	d	#	UB	Time	Fails	UB	Time	Nodes	Fails	LB	UB	Time	Fails	LB	UB	Time	Nodes	Fails	
60	0.1	50	4.0	0.0	0	4.0	0.0	63	0	4.0	4.0	0.2	0	4.0	4.1	0.1	3,639	4	
60	0.2	50	5.5	0.0	0	5.5	0.0	1,724	0	5.1	5.5	149.2	3	5.5	5.6	57.1	4,022	4	
60	0.3	50	7.0	0.0	0	7.0	0.0	6,986	0	7.0	7.0	34.3	1	7.0	7.9	286.8	25,256	35	
60	0.4	50	8.9	0.1	0	8.9	0.0	51,035	0	8.3	8.9	203.9	2	8.9	10.0	85.4	29,131	41	
60	0.5	50	10.7	0.2	0	10.7	0.1	212,856	0	10.1	10.7	106.0	1	10.7	11.8	150.5	25,552	33	
60	0.6	50	12.9	0.2	0	12.9	0.2	282,743	0	12.5	12.9	24.9	0	12.9	14.2	347.2	29,407	36	
60	0.7	50	15.6	0.2	0	15.6	0.2	214,544	0	15.3	15.6	9.7	0	15.6	16.6	186.5	17,808	21	
60	0.8	50	19.1	0.1	0	19.1	0.1	127,348	0	19.0	19.1	2.3	0	19.1	19.1	72.1	7,897	0	
60	0.9	50	25.7	0.0	0	25.7	0.0	5,457	0	25.7	25.7	0.2	0	25.7	25.7	4.2	725	0	
70	0.1	50	4.0	0.0	0	4.0	0.0	139	0	4.0	4.0	0.2	0	4.0	4.1	60.5	4,924	7	
70	0.2	50	6.0	0.0	0	6.0	0.0	2,717	0	5.8	6.0	36.5	0	6.0	6.8	0.8	15,644	39	
70	0.3	50	7.8	0.2	0	7.8	0.1	133,275	0	7.1	7.8	674.0	20	7.8	9.0	105.6	19,287	47	
70	0.4	50	9.7	1.3	0	9.7	1.0	1,395,809	0	9.1	9.7	523.5	14	9.7	11.2	429.2	21,300	43	
70	0.5	50	11.8	4.3	0	11.8	3.7	4,686,463	0	11.2	11.8	487.0	13	11.8	13.6	258.3	22,978	43	
70	0.6	50	14.1	6.5	0	14.1	5.5	6,456,304	0	13.6	14.1	174.2	4	14.1	16.1	600.3	24,455	39	
70	0.7	48	17.2	11.1	0	17.2	9.8	10,434,712	0	16.9	17.2	63.9	0	17.2	18.8	158.3	15,259	25	
70	0.8	50	21.4	2.0	0	21.4	2.3	2,283,368	0	21.2	21.4	14.4	0	21.4	21.8	182.7	29,335	8	
70	0.9	50	28.5	0.1	0	28.5	0.2	175,378	0	28.5	28.5	0.5	0	28.5	28.5	18.4	2,667	0	
75	0.1	49	4.0	0.0	0	4.0	0.0	290	0	4.0	4.0	17.9	0	4.0	4.1	74.4	3,108	5	
75	0.2	49	6.0	0.0	0	6.0	0.0	7,401	0	6.0	6.0	40.2	1	6.0	7.1	40.8	14,137	46	
75	0.3	49	8.0	0.2	0	8.0	0.1	207,328	0	7.5	8.0	202.2	12	8.0	9.4	42.2	16,213	48	
75	0.4	49	10.0	5.3	0	10.0	4.3	5,561,378	0	9.5	10.0	398.2	11	10.0	12.0	–	18,997	49	
75	0.5	49	12.1	27.6	0	12.1	23.6	27,595,210	0	11.8	12.1	182.1	4	12.1	14.4	802.7	20,161	41	
75	0.6	49	14.9	56.3	0	14.9	49.0	53,590,843	0	14.3	14.9	543.0	13	14.9	17.2	337.6	19,980	41	
75	0.7	49	18.0	59.7	0	18.0	50.4	50,638,728	0	17.6	18.0	146.2	2	18.0	19.6	264.0	13,849	24	
75	0.8	49	22.4	13.4	0	22.3	16.5	15,326,140	0	22.1	22.4	36.3	0	22.3	23.2	253.7	34,586	13	
75	0.9	49	1.0	1.2	0	29.9	2.6	2,376,532	0	29.9	1.0	23.3	0	29.9	30.0	74.6	11,929	1	
80	0.1	49	4.3	0.0	0	4.3	0.0	1,035	0	4.0	4.3	372.8	10	4.3	4.6	87.3	3,609	11	
80	0.2	49	6.3	0.1	0	6.3	0.1	95,440	0	6.0	6.3	431.3	15	6.1	7.4	1,161.8	11,427	48	
80	0.3	49	8.2	5.7	0	8.2	3.5	4,690,852	0	7.9	8.2	360.8	11	8.1	10.0	–	13,335	49	

best approaches. More details on the DIMACS benchmark library are given in the Appendix.

It is worth mentioning that the Branch-and-Price algorithms from [19] and [6] are specifically designed to tackle hard DIMACS instances. They rely on sophisticated techniques aiming at reducing the gap between the lower and the upper bounds. In the following, since DSATUR is the best algorithm for random instances, we decide to focus our analysis on this testbed only.

3. LOWER BOUNDS FOR THE VERTEX COLORING PROBLEM

In this section, we review the classical lower bounds for the VCP. In addition, we present a new bound based on a 1-to-1 mapping between VCPs and Stable Set Problems. For each bound we discuss the framework used to compute it and its computational complexity. The focus of this manuscript is to explore the idea of using these lower bounds to speed up the convergence of DSATUR. Accordingly, not only the strength of these bounds is important but also the computation time necessary to obtain them. Thus, we conclude this section with

an extensive computational comparison with special attention to their potential impact on the performance of DSATUR.

3.1. Lower Bounds Review

3.1.1. Clique Number $\omega(G)$. Recalling that a *clique* is a subset of fully connected vertices, the *clique number* $\omega(G)$ is the maximum size of a clique of G . The following holds:

$$\chi(G) \geq \omega(G). \quad (1)$$

This lower bound comes from the fact that in any clique all vertices should have different colors. Trivially any heuristically found clique of size $\omega^h(G)$ also provides a valid lower bound for the VCP ($\chi(G) \geq \omega^h(G)$). In our computation tests, $\omega^h(G)$ corresponds to the value of the heuristic clique produced by DSATUR_h. Computing the clique number is NP-hard (see Garey and Johnson [13]) but in practice very effective exact solvers are available in the literature. We address the interested reader to Wu and Hao [16] for a recent survey on the topic. In our computational tests, as mentioned in the previous section we decide to use the combinatorial Branch-and-Bound algorithm and Dynamic Programming based exact algorithm named Cliquer (see [25]).

3.1.2. Lower Bound Based on the Stability Number $\chi_\alpha(G)$. The *stability number* $\alpha(G)$ is the maximum size of a stable set of G . The following holds:

$$\chi(G) \geq \chi_\alpha(G) = \left\lceil \frac{n}{\alpha(G)} \right\rceil. \quad (2)$$

Since a coloring is a partition into stable sets, the best we can hope for is having all stable sets of maximum size (see Schrijver [29] for further details). Computing $\alpha(G)$ is an NP-hard problem (see Garey and Johnson [13]), but since $\alpha(G)$ is equivalent to the clique number $\omega(\bar{G})$ of the *complement graph* $\bar{G} = (V, \bar{E})$ where $\bar{E} = \{(i, j) : i, j \in V, i \neq j, (i, j) \notin E\}$. We use Cliquer (see [25]) to efficiently obtain it.

3.1.3. Fractional Coloring Number $\chi_f(G)$. Following the notation proposed in Schrijver [29], the Fractional Coloring number $\chi_f(G)$ is the minimum value of $\lambda_1 + \dots + \lambda_k$ with $\lambda_1, \dots, \lambda_k \in \mathbb{R}_+$ such that there exist stable sets $S_1, \dots, S_k$ with

$$\lambda_1 A^{S_1} + \dots + \lambda_k A^{S_k} = 1.$$

For any stable set S , A^S denotes the incidence vector of S in $\mathbb{R}^{|V|}$; that is for any $v \in G$:

$$A^S(v) := \begin{cases} 1 & \text{if } v \in S, \\ 0 & \text{otherwise.} \end{cases}$$

The Fractional Coloring number corresponds to the optimal solution value of the linear programming relaxation of the VCP formulation proposed by Mehrotra and Trick [21] and it is NP-hard to compute (see Grötschel et al. [14]). The following holds:

$$\chi(G) \geq \chi_f^*(G) = \lceil \chi_f(G) \rceil. \quad (3)$$

It is well-known that $\chi_f(G)$ provides strong VCP bounds and it requires Column Generation (CG) techniques to be computed. To obtain $\chi_f^*(G)$, we use the code available at <https://github.com/heldstephan/exactcolors>. The computational results in [6] show that this implementation is among the fastest in the literature.

3.1.4. Hoffman Number $\chi_H(G)$. Hoffman [17] proves that the following is a lower bound for $\chi(G)$:

$$\chi(G) \geq \chi_H^*(G) = \lceil \chi_H(G) \rceil = 1 - \frac{\varepsilon_{\max}(H)}{\varepsilon_{\min}(H)} \quad (4)$$

where H is the adjacency matrix of G while $\varepsilon_{\max}$ and $\varepsilon_{\min}$ are the largest and the smallest eigenvalues of H , respectively. The eigenvalues can be computed in polynomial time using the C++ LAPACK library.

3.2. A New Lower Bound

Cornaz and Jost [8] and Palubeckis [26] prove a 1-to-1 correspondence between colorings in G and stable sets in

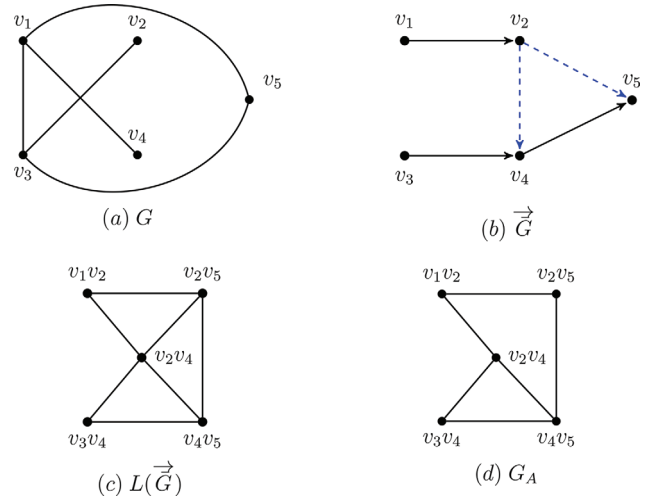


FIG. 1. Transformation from a graph G to G_A . [Color figure can be viewed at wileyonlinelibrary.com.]

an *auxiliary graph* $G_A = (V_A, E_A)$. The following theorem holds:

Theorem 1 (Cornaz and Jost [8]). *For any graph G and any acyclic orientation of its complementary graph, there is a one-to-one correspondence between the set of all colorings of G and the set of all stable sets of G_A . Moreover, for any coloring $\{V_1, \dots, V_k\}$ of G and its corresponding stable set $\tilde{S}$ in G_A , we have $|\tilde{S}| + k = |V|$. In particular:*

$$\alpha(G_A) + \chi(G) = |V|.$$

To build the auxiliary graph G_A , it is necessary to define an acyclic orientation $\vec{G}$ of $\bar{G}$. Then G_A corresponds to the line-graph¹ $L(\vec{G})$ after the removal of all edges between pairs of arcs which are simplicial² in $\vec{G}$. Precisely, given a simplicial pair of arcs $a = (v_i, v_j)$ and $b = (v_i, v_k)$ the corresponding edge (a, b) is removed from $L(\vec{G})$.

We now illustrate the construction of G_A using the example of Figure 1. The original graph consists of five nodes and five edges (Fig. 1a). Then the acyclic orientation $\vec{G}$ is depicted in Figure 1b where $(v_i, v_j) \in \vec{E}$ if $(v_i, v_j) \in \bar{E}$ and $i < j$. The next step consists of creating the line-graph $L(\vec{G})$ as depicted in Figure 1c. Finally, the Auxiliary Graph G_A is given in Figure 1d. Only one simplicial pair (dashed lines) is present in $\vec{G}$, i.e., (v_2, v_5) and (v_2, v_4) , and accordingly the corresponding edge has been removed from $L(\vec{G})$. From

¹ The line-graph $L(\vec{G})$ of $\vec{G}$ is defined as follows: each arc of $\vec{G}$ corresponds to a vertex of $L(\vec{G})$ and two vertices are linked by an edge in the $L(\vec{G})$ if they correspond to two adjacent arcs in $\vec{G}$.

² A pair of arcs $\{a, b\}$ of $\vec{G}$ is called a simplicial pair if $a=(u,v)$, $b=(u,w)$, and (v,w) or (w,v) is an arc of $\vec{G}$, for three distinct vertices u, v, w .

Figure 1, it is clear that any vertex belonging to a stable set in G_A allows one to reduce the upper bound $|V|$ by one unit. In other words, if a vertex $(v_i v_j) \in G_A$ belongs to a stable set, it means that vertex v_j can be colored with the same color as v_i , i.e., “saving” in this manner a color. Finally, for any simplicial pair of arcs (v_i, v_j) and (v_i, v_k) in $\vec{G}$, removing the arc $(v_i v_j, v_i v_k)$ in G_A reflects the fact that once v_k has been colored in the same way as v_i then also v_j can take the same color (and vice-versa).

Any upper bound $\bar{\alpha}(G_A)$ on the stability number $\alpha(G_A)$ gives us a valid lower bound for $\chi(G)$ denoted $\chi_{G_A}(G)$. The following holds:

$$\chi(G) \geq \chi_{G_A}(G) = |V| - \lfloor \bar{\alpha}(G_A) \rfloor. \quad (5)$$

Thanks to extensive computational tests, we identify that the best compromise between the quality of the upper bound $\bar{\alpha}(G_A)$ and the computation time can be achieved solving a relaxation of the *edge formulation* for the Maximum Stable Set problem (MSSP). The edge formulation is an ILP where $\alpha(G_A) = \max x$ over x in $STAB(G_A)$, that is, the set of vectors of $\mathbb{R}^{|V_{G_A}|}$ satisfying

$$x_u + x_v \leq 1 \quad (u, v) \in E_A \quad (6)$$

$$x_v \in \{0, 1\} \quad v \in V_A. \quad (7)$$

To solve this formulation, commercial ILP solvers start by computing the *fractional stable set value*, i.e., the optimal solution value of the linear programming relaxation. Then general purpose valid inequalities (*cuts*) are heuristically added to strengthen the bound before branching. The strength of this bound depends on the solver used and also on the policies designed to generate cuts at the root node. Thanks to preliminary computational tests, the bound computed by CPLEX version 12.6 (with default parameters and stopping the optimization at the root node) proves to be a very strong and fast bound for $\alpha(G_A)$. It is worth mentioning that we also tested different ways of obtaining $\alpha(G_A)$ starting from the fractional stable set number. More precisely, we tried to heuristically separate a subset of the Clique Inequalities or to exactly separate the Odd Cycle Inequalities, but the resulting bounds $\chi_{G_A}(G)$ are very weak.

We perform additional tests to check the possibility of obtaining a better compromise between strength and computing time, tuning the parameters of CPLEX. Thanks to extensive computational experiments, we identify that the only effective families of cuts are the *Clique Cuts*, the *Zero-half Cuts*, and the *Gomory Fractional Cuts*. Since the cuts are heuristically added, repeating the generation several times can potentially produce better bounds. To check if better bounds are possible, we repeat the default cut generation of CPLEX up to five times. More than 90% of the times, the best bound value is obtained with only one iteration. We conclude that it is not worth repeating the cut generation more than once. We finally try to limit the amount of time of the cut generation of CPLEX, but the quality of the bound largely deteriorates. Summarizing, the best CPLEX

parameter configuration setting to compute χ_{G_A} is default, switching off the generation of all families of cuts except the *Clique Cuts*, the *Zero-half Cuts*, and the *Gomory Fractional Cuts*. Finally, to avoid rounding errors possibly due to numerical imprecisions, we use the following formula to compute $\lfloor \bar{\alpha}(G_A) \rfloor = \lfloor (\kappa + \varepsilon) \rfloor$, where $\varepsilon = 10^{-3}$ and κ is the upper bound value returned by CPLEX. Since we are only interested in the upper bound value, we switch off all the routines of CPLEX to compute heuristic solutions. The quality and the computing time of this bound is discussed in the next section and compared to the other bounds.

3.3. Comparison Between Lower Bounds

To compare and test the lower bounds, we use the same benchmark of random instances introduced in San Segundo [27] and already used in Section 2. Each line in Table 2 reports average results in the same fashion of Table 1. Table 2 presents the average bound values and the average computation times for χ_f^* , χ_{G_A} , ω , χ_α , χ_H^* , and ω^h .

In Table 2, we computationally find that the strongest lower bound values are provided by χ_{G_A} and χ_f^* . Conversely, these two bounds are also the most time consuming ones. Among the fast but weaker lower bounds, ω clearly dominates χ_α , χ_H^* , and ω^h in terms of lower bound values.

As far as the strongest bounds are concerned, χ_{G_A} is equal to χ_f^* in most of the instances and, in the remaining cases, the difference between the values does not exceed 1. Thanks to extensive computational tests, we observe that the quality of the bounds is crucial to prune the branching nodes of DSATUR. We can then discard all the bounds except χ_{G_A} and χ_f^* from the analysis.

The two subfigures of Figure 2 graphically compare the lower bound values and the computation time of χ_{G_A} and χ_f^* for instances of 70 vertices and density varying from 0.1 to 0.9. Figure 2a presents the lower bound values on the vertical axis and the density on the horizontal one (50 instances per density). Figure 2b presents the lower bound computation times on the vertical axis (logarithmic scale) and the density on the horizontal one. The symbol “+” represents the results of χ_f^* and the symbol “□” represents the results of χ_{G_A} .

In Figure 2a, we can graphically see that the maximum difference between χ_f^* and χ_{G_A} is at most 1. For $d = 0.1, 0.2, 0.3, 0.4, 0.5, 0.6$, χ_f^* is much faster than χ_{G_A} as shown in Figure 2b. From $d = 0.7$, the difference between the computation times gradually reduces.

In Table 3, we report the same information as for Table 2 but only for χ_{G_A} and χ_f^* . For $n = 85$ and $n = 90$, the average value of χ_{G_A} is better than the average value of χ_f^* (similar computation times and χ_{G_A} is slightly faster for $n = 90$). For $n = 85$ the average value of χ_{G_A} and χ_f^* coincide (similar computation times).

Figure 3 is similar to Figure 2b but it focuses on instances with $d = 0.7, 0.8, 0.9$. For $d = 0.7, 0.8$, the computation time of χ_{G_A} is of the same order of magnitude as the computation time of χ_f^* . For density $d = 0.9$, χ_{G_A} becomes faster

TABLE 2. Comparison between different lower bounds for random instances

Instance			Average values						Average times					
n	d	#	χ_f^*	χ_{G_A}	ω	χ_α	χ_H^*	ω^h	χ_f^*	χ_{G_A}	ω	χ_α	χ_H^*	ω^h
60	0.1	50	4.00	4.00	3.36	2.00	2.00	1.32	0.03	1.56	0.00	0.00	0.00	0.00
60	0.2	50	5.06	5.00	4.46	3.18	2.98	1.60	0.07	3.24	0.00	0.00	0.00	0.00
60	0.3	50	6.98	6.18	5.48	4.74	3.34	1.94	0.08	2.77	0.00	0.00	0.00	0.00
60	0.4	50	8.28	7.96	6.46	5.80	4.00	1.92	0.07	2.01	0.00	0.00	0.00	0.00
60	0.5	50	10.12	9.98	8.02	7.02	4.88	2.42	0.06	1.32	0.00	0.00	0.00	0.00
60	0.6	50	12.50	12.16	9.70	8.76	5.80	2.86	0.05	0.62	0.00	0.00	0.00	0.00
60	0.7	50	15.30	15.24	12.34	11.20	6.50	3.54	0.05	0.26	0.00	0.00	0.00	0.00
60	0.8	50	19.04	19.10	16.14	13.92	8.02	5.80	0.04	0.11	0.00	0.00	0.00	0.00
60	0.9	50	25.66	25.66	23.56	18.30	11.18	9.04	0.01	0.00	0.00	0.00	0.00	0.00
70	0.1	50	4.00	4.00	3.60	2.00	2.10	2.10	0.07	4.24	0.00	0.00	0.00	0.00
70	0.2	50	5.84	5.04	4.60	3.68	3.00	2.04	0.16	9.49	0.00	0.00	0.00	0.00
70	0.3	50	7.10	6.62	5.60	4.98	3.68	2.22	0.14	6.70	0.00	0.00	0.00	0.00
70	0.4	50	9.10	8.34	7.02	6.74	4.26	2.46	0.11	4.94	0.00	0.00	0.00	0.00
70	0.5	50	11.18	10.62	8.34	7.82	5.04	2.60	0.09	3.10	0.00	0.00	0.00	0.00
70	0.6	50	13.64	13.18	10.22	10.00	6.04	2.94	0.08	1.53	0.00	0.00	0.00	0.00
70	0.7	48	16.88	16.73	12.90	11.94	7.19	3.42	0.07	0.47	0.00	0.00	0.00	0.00
70	0.8	50	21.22	21.30	17.32	14.96	8.80	5.04	0.06	0.17	0.00	0.00	0.00	0.00
70	0.9	50	28.50	28.52	25.60	19.76	12.84	8.94	0.02	0.02	0.00	0.00	0.00	0.00
75	0.1	49	4.00	4.00	3.61	2.10	2.10	1.31	0.09	7.44	0.00	0.00	0.00	0.00
75	0.2	49	6.00	5.08	4.59	3.78	3.00	1.63	0.20	15.12	0.00	0.00	0.00	0.00
75	0.3	49	7.53	6.86	5.73	5.06	3.88	1.61	0.18	9.75	0.00	0.00	0.00	0.00
75	0.4	49	9.47	8.59	7.02	6.63	4.06	2.16	0.14	7.41	0.00	0.00	0.00	0.00
75	0.5	49	11.80	10.98	8.61	8.59	5.00	2.45	0.12	4.84	0.00	0.00	0.00	0.00
75	0.6	49	14.29	13.92	10.41	9.98	5.98	2.71	0.10	2.24	0.00	0.00	0.00	0.00
75	0.7	49	17.63	17.49	13.33	12.47	7.37	3.49	0.09	0.67	0.00	0.00	0.00	0.00
75	0.8	49	22.10	22.08	17.67	15.92	9.02	5.16	0.08	0.21	0.00	0.00	0.00	0.00
75	0.9	49	29.92	29.92	26.39	19.86	13.53	7.69	0.03	0.03	0.00	0.00	0.00	0.00
80	0.1	49	4.02	4.00	3.76	2.22	2.06	2.02	0.13	11.24	0.00	0.01	0.00	0.00
80	0.2	49	6.00	5.14	4.82	3.98	3.00	2.14	0.25	27.63	0.00	0.00	0.00	0.00
80	0.3	49	7.94	6.98	5.84	5.41	3.92	2.06	0.22	14.36	0.00	0.00	0.00	0.00

than χ_f^* in most of the instances. According to the construction of graph G_A , the more dense the graph the faster the bound is computed, since the number of vertices $|V_A|$ of G_A is equal to the number of edges of $\tilde{G}$. While it gets faster, it preserves its quality and, accordingly, its pruning potential once included in DSATUR. Furthermore, the bound χ_{G_A} can be computed in polynomial time (depending on the effort for computing an upper bound on the MSSP), while the fractional chromatic number requires solving maximum weight stable set problems exactly (then it is NP-hard to compute; see Section 3.1).

Recalling that the goal of this research is to introduce a bounding technique for DSATUR and considering the large number of nodes explored on average by this algorithm (see Table 1), only an extremely fast and strong bound can be used. Unfortunately the current state of the art does not allow one to compute fast and strong bounds for medium and low density random graphs using χ_{G_A} . Accordingly, we are forced to discard from the analysis all the instances with density $d < 0.9$. Within the nodes of DSATUR, the bounds are actually computed starting from a partial coloring. The impact of increasing the number of colored vertices is to create smaller and denser graphs G_A (see the next section for further details). Accordingly, the bound χ_{G_A} becomes the perfect candidate for our bounding technique.

In the Appendix, we report the lower bounds for the DIMACS instances. We can observe that the overall quality of χ_α , χ_H^* , and ω^h is very poor, while χ_{G_A} and χ_f^* provide the best lower bound for many instances. In the DIMACS benchmark set there are six instances with density $d \geq 0.9$: DSJC1000.9, DSJC125.9, DSJC250.9, DSJC500.9, DSJR500.1c, r1000.1c.

In Table 4, we report the same information as for Table 2. As far as the comparison between χ_f^* and χ_{G_A} is concerned, we can see that in three out of six instances the values of the two bounds coincide (similar computation times). For one instance, both bounds cannot be computed within the time limit of 1,500 s. For the remaining two instances, i.e., DSJC500.9 and r1000.1c, the value of χ_f^* is greater than the value of χ_{G_A} and for DSJC500.9, χ_f^* can be obtained in shorter computation time while, for r1000.1c in larger computation time. It is worth noticing that for four of six instances, the bound ω cannot be computed within the time limit.

4. REDUCED GRAPH AND THE IMPROVED DSATUR-BASED BRANCH-AND-BOUND ALGORITHM

To make lower bounds dependent on a partial coloring $\tilde{C}$ obtained during the execution of DSATUR, we introduce

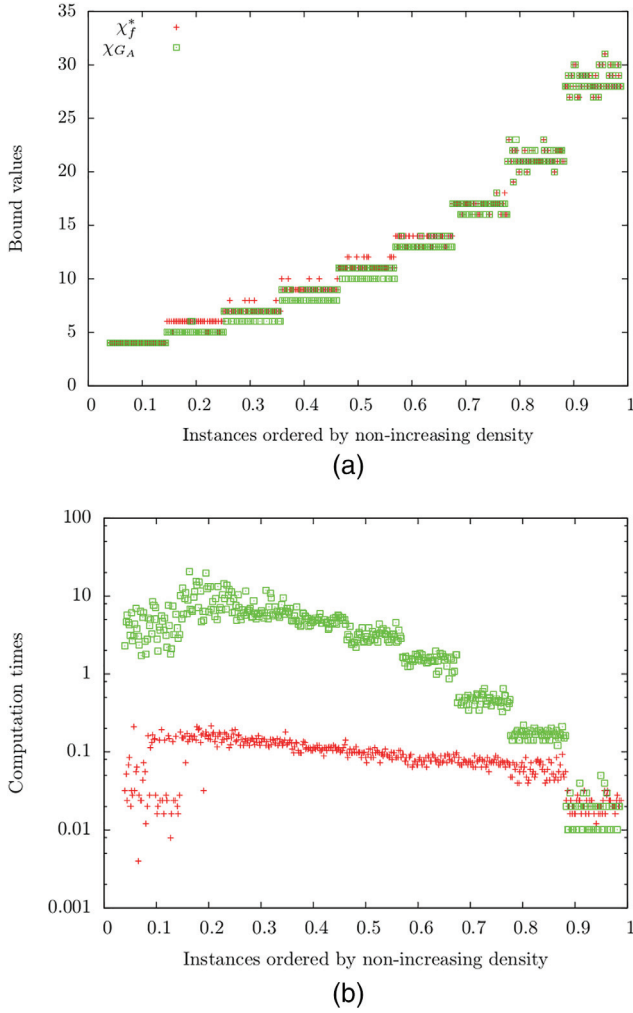


FIG. 2. Lower bound comparison for random instances with different densities and $n = 70$. [Color figure can be viewed at wileyonlinelibrary.com.]

TABLE 3. Comparison between χ_f^* and χ_{GA} for random instances with $n = 80, 85, 90$ and $d = 0.9$.

Instance			Average values		Average times	
n	d	#	χ_f^*	χ_{GA}	χ_f^*	χ_{GA}
80	0.9	50	31.28	31.30	0.03	0.03
85	0.9	50	32.80	32.80	0.04	0.04
90	0.9	50	34.12	34.14	0.06	0.05

a new graph. This *Reduced Graph* $G^{\tilde{C}} = (V^{\tilde{C}}, E^{\tilde{C}})$ is composed of the sub-graph of G induced by the uncolored vertices plus $\tilde{k}$ vertices, one for each color. Each new vertex $\tilde{v}_i$, representing color i , is connected to all the uncolored neighbors of the vertices of V_i and to all the others $\tilde{k} - 1$ new vertices. Thus, the subgraph of $G^{\tilde{C}}$ induced by the $\tilde{k}$ new vertices is a clique. The reduced graph becomes smaller increasing the number of colored vertices thus also the lower bounds become easier to compute. An example using a partially colored graph of six vertices is given in Figure 4, where

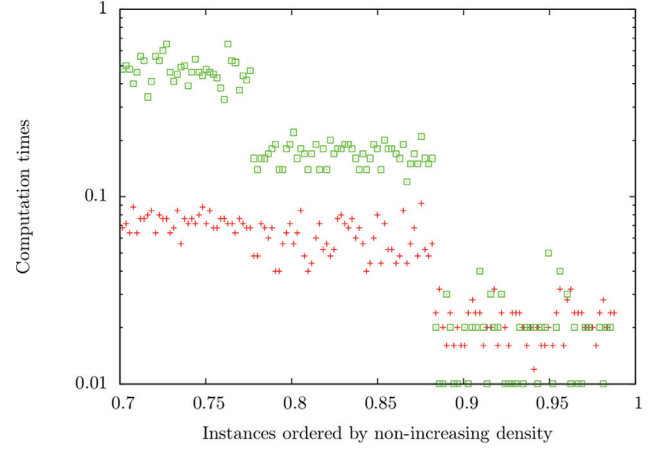


FIG. 3. Lower bound comparison for random instances with $d = 0.7, 0.8, 0.9$ and $n = 70$. [Color figure can be viewed at wileyonlinelibrary.com.]

two colors (1 and 2) are used and three vertices are uncolored. The Reduced Graph has five vertices, two representing the classes of colors plus the three original uncolored vertices.

Recalling that we denote by $\chi_{\tilde{C}}(G)$ the chromatic number of G partially colored by $\tilde{C}$, the following holds:

Lemma 1. $\chi_{\tilde{C}}(G) = \chi_{\tilde{C}}(G^{\tilde{C}})$

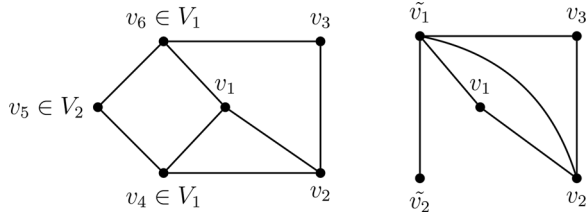
Proof. Any feasible coloring $\hat{C} = \{\hat{C}_1, \dots, \hat{C}_k\}$ in G containing the coloring $\tilde{C}$ (inducing $k \geq \tilde{k}$) can be mapped into a unique feasible coloring $\{\hat{C}'_1, \dots, \hat{C}'_k\}$ in $G^{\tilde{C}}$ using the same number of colors k . For each color i , $i = 1, \dots, k$, two cases arise: if $\hat{C}_i \subseteq V \setminus \tilde{C}$, then $\hat{C}'_i \leftarrow \hat{C}_i$ (since the subgraphs induced by $\hat{C}_i$ are equivalent in G and $G^{\tilde{C}}$); otherwise there exists a unique stable set j such that $\tilde{C}_j \subseteq \hat{C}_i$, then $\hat{C}'_i \leftarrow \{\tilde{v}_j\} \cup \{\hat{C}_i \setminus \tilde{C}_j\}$ with $\tilde{v}_j$ representing $\tilde{C}_j$ in $G^{\tilde{C}}$. By construction, $\hat{C}'_i$ is a stable set $G^{\tilde{C}}$. Any feasible coloring $\{\hat{C}'_1, \dots, \hat{C}'_k\}$ in $G^{\tilde{C}}$ can be mapped into a unique feasible coloring $\{\hat{C}_1, \dots, \hat{C}_k\}$ containing the coloring $\tilde{C}$ in G . For each color i , $i = 1, \dots, k$, two cases arise: if $\hat{C}'_i \subseteq V \setminus \tilde{C}$, then $\hat{C}_i \leftarrow \hat{C}'_i$; otherwise a unique vertex $\tilde{v}_j$ representing $\tilde{C}_j$ belongs to $\hat{C}'_i$ (let us recall that the $\tilde{k}$ vertices representing $\tilde{C}$ in $G^{\tilde{C}}$ form a clique), and then $\hat{C}_i \leftarrow \tilde{C}_j \cup \{\hat{C}'_i \setminus \{\tilde{v}_j\}\}$. By construction, $\hat{C}_i$ is a stable set in G . ■

Trivially, we have: $\chi_{\tilde{C}}(G^{\tilde{C}}) = \chi(G^{\tilde{C}})$. Thus, from Lemma 1, a lower bound denoted by $LB^{\tilde{C}}$ for $\chi(G^{\tilde{C}})$ is a lower bound for $\chi_{\tilde{C}}(G)$.

We present now the *improved DSATUR-based Branch-and-Bound algorithm*. The new DSATUR algorithm, denoted $DSATUR_{\chi_{GA}}$, is obtained by replacing in Algorithm 1 the call to Algorithm 2 by a call to the new Algorithm 3. The key element of the new algorithm is the introduction of the new lower bound $\chi_{GA}(G^{\tilde{C}})$, computed thanks to the reduced graph $G^{\tilde{C}}$ at nodes of the branching tree.

TABLE 4. Comparison between lower bounds on $\chi(G)$ for DIMACS instances.

Instance				Value						Time					
Name	n	m	d	χ_f^*	χ_{GA}	ω	χ_α	χ_H^*	ω^h	χ_f^*	χ_{GA}	ω	χ_α	χ_H^*	ω^h
DSJC1000.9	1,000	449,449	0.9	–	–	–	166	48	4	tl	tl	tl	0.0	0.3	0.0
DSJC125.9	125	6,961	0.9	43	43	34	31	16	10	0.2	0.2	10.2	0.0	0.0	0.0
DSJC250.9	250	27,897	0.9	71	71	–	50	23	7	3.6	5.6	tl	0.0	0.0	0.0
DSJC500.9	500	112,437	0.9	123	122	–	100	33	11	50.1	1,060.79	–	0	0.06	0
DSJR500.1c	500	121,275	≈ 1.0	85	85	12	38	25	2	10.3	2.0	0.0	0.0	0.1	0.0
r1000.1c	1,000	485,090	≈ 1.0	96	95	–	41	25	8	1,239.8	403.1	tl	0.0	0.3	0.0

FIG. 4. A partially colored graph G and the Reduced Graph $G^{\tilde{C}}$.

Since updating the lower bound can be computationally expensive, we propose strategies to compute it only in “promising” nodes of the branching tree. The term “promising” is linked to two different aspects. Clearly, the chances to prune a node are higher when the incumbent solution value UB is close to the chromatic number $\chi(G)$. Similarly, the lower bound is more likely to prune a node when the difference between the node lower bound ($\tilde{k}$) and upper bound (UB) is small. Accordingly, we propose a function $\phi_{u,g}(\tilde{C})$ which controls the computation of χ_{GA} according to two predefined parameters u and g . The first parameter u corresponds to the number of times the incumbent solution value UB has been updated and it is then a measure of the quality of current incumbent value UB . The second parameter g corresponds to the gap between the incumbent value UB and the lower bound $\tilde{k}$ and it is related to the probability that the bound χ_{GA} can prune a specific node.

In Algorithm 3, the lower bound $LB(\tilde{C})$ associated with the coloring $\tilde{C}$ of a given node is initialized with the maximum between the global lower bound LB and the current number of colors $\tilde{k}$. If the function $\phi_{u,g}(\tilde{C})$ returns true, the computation of $\chi_{GA}(G^{\tilde{C}})$ is performed to improve the value of $LB(\tilde{C})$. Then, if $LB(\tilde{C})$ is greater than the best incumbent solution value UB , the children nodes of the current node are pruned. The rest of the algorithm remains unchanged.

We implemented two additional improvements designed to enhance the performances of $DSATUR_{\chi_{GA}}$. The first one concerns the cut separation performed by CPLEX at the root node used to compute $\bar{\alpha}(G_A)$ (see Section 3.2). The separation is interrupted as soon as $\lfloor \bar{\alpha}(G_A) \rfloor \geq \tilde{n} - UB$ (implemented thanks to the CPLEX cutcallback). This improvement is particularly important since it allows pruning nodes as soon as the value of the bound is strong enough, avoiding in this manner unnecessary computational efforts. The

Algorithm 3 $DSATUR_{\chi_{GA}}(\tilde{C})$

```

1 if all the vertices are colored then
2   if  $\tilde{k} < UB$  then
3      $C^* \leftarrow \tilde{C}$ ,  $UB \leftarrow \tilde{k}$ ;
4   end
5 else
6    $LB(\tilde{C}) \leftarrow \max\{LB, \tilde{k}\}$ ;
7   if  $\phi_{u,g}(\tilde{C}) = \text{true}$  then
8      $LB(\tilde{C}) \leftarrow \max\{LB(\tilde{C}), \chi_{GA}(G^{\tilde{C}})\}$ ;
9     if  $LB(\tilde{C}) \geq UB$  then
10       return
11     end
12     select an uncolored vertex  $v$ ;
13     for every feasible color  $i \in \tilde{C}$  plus a new one do
14        $\hat{C} \leftarrow \tilde{C}$ , add  $v$  in  $\hat{V}_i$ ;
15       if  $\max\{\tilde{k}, LB(\tilde{C})\} < UB$  then
16          $DSATUR_{\chi_{GA}}(\hat{C})$ ;
17       end
18     end
19   end
20 end

```

second improvement concerns the initialization of UB using the upper bound provided by the feasible coloring of $DSATUR_h$. The rationale behind this choice is to improve the quality of the initial solution to increase the chances to prune the nodes in the initial phase of the algorithm, i.e., before high quality (optimal) solutions are found. The performance of $DSATUR_{\chi_{GA}}$ is discussed in the next section.

During an execution of $DSATUR_{\chi_{GA}}$, it is possible that two reduced graphs $G^{\tilde{C}_1}$ and $G^{\tilde{C}_2}$ associated with two particular branching nodes and two particular partial colorings $\tilde{C}_1$ and $\tilde{C}_2$ are isomorphic and thus $\chi_{GA}(G^{\tilde{C}_1}) = \chi_{GA}(G^{\tilde{C}_2})$. In these cases, $\chi_{GA}(G^{\tilde{C}})$ does not need to be computed again. While the complexity of graph isomorphism is an open question, we can still recognize two similar graphs in specific situations in $DSATUR_{\chi_{GA}}$, e.g., when a vertex is colored with a new color and each stable set of the partial coloring contains at least one vertex of its neighbourhood.

TABLE 5. DSATUR and DSATUR $_{\chi_{G_A}}$ for random instances with high density.

Instance			DSATUR reimplementation [10]				DSATUR $_{\chi_{G_A}}$				
n	d	#	UB	Time	Nodes	Fails	UB	Time	Nodes	#bounds * \ #bounds	Fails
60	0.9	50	25.66	0.00	5,457	0	25.66	0.15	65	3.10 \ 62.80	0
70	0.9	50	28.52	0.18	175,378	0	28.52	0.37	123	8.08 \ 119.64	0
75	0.9	49	29.94	2.65	2,376,532	0	29.94	0.53	153	11.59 \ 149.10	0
80	0.9	50	31.34	5.25	3,369,599	0	31.34	0.74	177	15.38 \ 173.38	0
85	0.9	50	32.80	20.21	12,244,647	0	32.80	1.02	211	18.44 \ 206.88	0
90	0.9	50	34.16	112.03	62,885,746	0	34.14	1.52	267	28.68 \ 261.70	0
95	0.9	5	35.20	264.45	153,574,080	0	35.20	1.64	263	25.40 \ 258.00	0
100	0.9	5	37.60	846.01	623,601,052	4	37.00	3.59	349	39.20 \ 343.40	0
105	0.9	5	39.00	586.74	426,267,090	2	38.20	3.19	358	39.80 \ 352.20	0
110	0.9	5	40.40	–	597,603,410	5	39.20	7.05	495	67.00 \ 488.00	0
120	0.9	5	43.40	–	537,554,211	5	41.60	14.28	570	106.20 \ 563.20	0
130	0.9	5	48.00	–	556,669,663	5	43.80	72.27	1,189	339.00 \ 1,178.60	0
140	0.9	5	51.20	–	481,086,785	5	47.20	57.61	1,072	262.80 \ 1,062.60	0

5. COMPUTATIONAL RESULTS

All the algorithms are coded in C/C++, and run on a PC with an Intel(R) Core(TM) i7-4770 CPU at 3.40GHz and 16 GB RAM memory, under Linux Ubuntu 14.04 64-bit. As discussed in Section 3.3, χ_{G_A} can be computed efficiently just for high density graphs ($d = 0.9$), thus we focus our computation results on these instances. For lower density instances, DSATUR outperforms DSATUR $_{\chi_{G_A}}$ in terms of computing time. This is due to the fact that the bounding technique based on χ_{G_A} reduces only the number of nodes while the additional computational effort is higher than the time saved cutting parts of the search tree. The node processing time of DSATUR is extremely low since only the operations linked to the selection of the next vertex to color are executed. Accordingly, bounding techniques based on weaker bounds (e.g., ω) or strong but time-consuming bounds (e.g., χ_{G_A} for $d < 0.9$) only allow a reduction of the number of nodes but they increase the overall computing time.

To demonstrate the performance of DSATUR $_{\chi_{G_A}}$, we propose the following testbed. A first part is composed of the high density random instances of [27] with $n = 60, 70, 75$ and $d = 0.9$. A second part is composed of larger high density random graphs. To fully test the potential of DSATUR $_{\chi_{G_A}}$, we generate 50 additional instances for each graph size $n = 80, 85, 90$ and only five additional instances for larger graphs with $n = 95, 100, 110, 120, 130, 140$ with $d = 0.9$.

The goal of this computation section is twofold. First, we test the impact of the proposed bounding procedure in Subsection 5.1 updating the lower bound at each node of the branching scheme. Then we discuss possible enhancements based on the function ϕ to select a promising subset of nodes in which the lower bound is computed (Subsection 5.2).

5.1. Computing Lower Bounds at Each Node of the Branching Scheme

In this section, we compute χ_{G_A} at each node of the branching tree to test the impact in reducing the number of explored

nodes. Accordingly, in Algorithm 3, the function $\phi_{u,g}(\tilde{C})$ always returns true. Table 5 reports the results on the new test bed of instances. The table is divided into three parts: in the first part we present the instance features, in the second part we present the results obtained by DSATUR, and in the third part we present the results obtained by DSATUR $_{\chi_{G_A}}$. The same time limit of 1,200 s is imposed. The following average information is reported for each algorithm:

- *UB*: the average number of colors (the chromatic number or an upper bound in case time limit is reached).
- *nodes*: the average number of explored nodes.
- *time*: the average computation time (in seconds).
- *#bounds*: the average number of times χ_{G_A} is computed.
- *#bounds**: the average number of times that χ_{G_A} is able to prune.
- *fails*: the number of instances that cannot be solved within the time limit.

Table 5 clearly shows that DSATUR $_{\chi_{G_A}}$ drastically reduces the number of explored nodes. For $n = 60, 70, 80$, this reduction is significant but DSATUR remains the fastest algorithm. For $n = 75, 85, 90, 95$, DSATUR $_{\chi_{G_A}}$ becomes the fastest algorithm. For $n = 100, 105$, DSATUR is not able to solve 4 and 2 instances, respectively, and starting from $n = 110$ no instances can be solved to proven optimality in less than 1,200 s by DSATUR. All these instances can be solved instead by DSATUR $_{\chi_{G_A}}$ in short computing time.

Among the hard DIMACS instances, only six have a density $d \geq 0.9$. None of them can be exactly solved by DSATUR [27] while only DSJR500.1c can be solved via a Branch-and-Price algorithm (see the Appendix). DSATUR $_{\chi_{G_A}}$ is able to solve DSJC125.9 in 26.41 s and 680 nodes. Among the 672 bounds computed, 128 bounds successfully prune the node.

In Table 5, we can see that only a fraction of the calculated bounds (*#bounds*) are able to prune the nodes (*#bounds**). This fact suggests that better performance can be achieved using the function $\phi_{u,g}(\tilde{C})$, i.e., trying to limit the number of useless bound computations. In addition, since the

TABLE 6. DSATUR $_{\chi_{GA}}$ and DSATUR $_{\chi_{GA}}$ with different functions $\phi_{u,g}$.

Instance			$\phi_{0,0}$		$\phi_{0,2}$		$\phi_{0,3}$		$\phi_{0,4}$		$\phi_{1,1}$		$\phi_{2,1}$		$\phi_{3,1}$	
n	d	#	Time	Fails	Time	Fails	Time	Fails	Time	Fails	Time	Fails	Time	Fails	Time	Fails
60	0.9	50	0.15	0	0.11	0	0.08	0	0.05	0	0.05	0	0.00	0	0.01	0
70	0.9	50	0.37	0	0.30	0	0.24	0	0.18	0	0.19	0	0.14	0	0.12	0
75	0.9	49	0.53	0	0.44	0	0.36	0	0.33	0	0.28	0	0.24	0	0.27	0
80	0.9	50	0.74	0	0.51	0	0.44	0	0.43	0	0.64	0	0.35	0	1.01	0
85	0.9	50	1.02	0	0.75	0	0.65	0	0.64	0	0.87	0	0.57	0	2.36	0
90	0.9	50	1.52	0	1.12	0	1.02	0	1.35	0	1.32	0	1.74	0	11.79	0
95	0.9	5	1.64	0	1.41	0	1.36	0	1.45	0	0.85	0	0.81	0	0.89	0
100	0.9	5	3.59	0	3.24	0	10.37	0	49.38	0	1.98	0	1.93	0	2.62	0
105	0.9	5	3.19	0	2.88	0	3.16	0	6.19	0	1.58	0	1.54	0	2.95	0
110	0.9	5	7.05	0	7.45	0	22.56	0	147.32	0	4.62	0	4.56	0	5.18	0
120	0.9	5	14.28	0	13.68	0	11.91	1	25.98	1	10.17	0	10.09	0	29.05	0
130	0.9	5	72.27	0	123.70	0	33.24	1	120.05	1	55.34	0	55.23	0	62.36	0
140	0.9	5	57.61	0	56.50	0	57.15	0	88.65	0	39.37	0	39.47	0	51.88	0

TABLE 7. DSATUR and DSATUR $_{\chi_{GA}}$.

Instance			DSATUR reimplementation [10]				DSATUR $_{\chi_{GA}}$ with $\phi_{2,1}$				
n	d	#	UB	Time	Nodes	Fails	UB	Time	Nodes	#bounds * \ #bounds	Fails
60	0.9	50	25.66	0.00	5,457	0	25.66	0.00	293	3.02 \ 8.82	0
70	0.9	50	28.52	0.18	175,378	0	28.52	0.14	405	16.24 \ 55.56	0
75	0.9	49	29.94	2.65	2,376,532	0	29.94	0.24	472	22.31 \ 82.04	0
80	0.9	50	31.34	5.25	3,369,599	0	31.34	0.35	26,313	27.26 \ 101.00	0
85	0.9	50	32.80	20.21	12,244,647	0	32.80	0.57	79,936	34.60 \ 134.90	0
90	0.9	50	34.16	112.03	62,885,746	0	34.14	1.74	684,422	45.66 \ 183.76	0
95	0.9	5	35.20	264.45	153,574,080	0	35.20	0.81	22,346	49.80 \ 185.40	0
100	0.9	5	37.60	846.01	623,601,052	4	37.00	1.93	7,818	65.40 \ 266.40	0
105	0.9	5	39.00	586.74	426,267,090	2	38.20	1.54	998	65.80 \ 277.40	0
110	0.9	5	40.40	—	597,603,410	5	39.20	4.56	960	96.00 \ 404.60	0
120	0.9	5	43.40	—	537,554,211	5	41.60	10.09	41,070	131.80 \ 459.40	0
130	0.9	5	48.00	—	556,669,663	5	43.80	55.23	1,599	376.80 \ 1,055.60	0
140	0.9	5	51.20	—	481,086,785	5	47.20	39.47	75,874	298.80 \ 913.80	0

computational effort to compute the bounds represents the majority of the time of DSATUR $_{\chi_{GA}}$ (on average, 86% of the total time), finding effective strategies to trigger the computation only in promising nodes becomes a crucial issue. In the following section we discuss the proposed strategies for $\phi_{u,g}(\tilde{C})$ and their impact on the performance of DSATUR $_{\chi_{GA}}$.

5.2. Computing Lower Bounds Only at Specific Nodes of the Branching Scheme

Given two input parameter u and g , the function $\phi_{u,g}$ is defined as follows:

$$\phi_{u,g}(\tilde{C}) = (u^* \geq u \text{ AND } \tilde{g} \geq g)$$

where u^* corresponds to the number of times the incumbent solution UB has been updated and, $\tilde{g} = UB - \tilde{k}$ (a measure of the difference between the node lower and upper bound). If $u = 0$ and $g = 0$, the corresponding function $\phi_{0,0}(\tilde{C})$ always returns true, and it corresponds to the case in which the bound is computed at each node of the tree.

The results of DSATUR $_{\chi_{GA}}$ with different functions $\phi_{u,g}$ are reported in Table 6. We test different values of the parameters u and g . For parameter u we test 1, 2, 3 while for the parameter g we test 2, 3, 4. DSATUR $_{\chi_{GA}}$ with $\phi_{1,1}, \phi_{2,1}, \phi_{3,1}$ clearly outperforms the results obtained with $\phi_{0,g}$ with $g \geq 2$. With the parameter $u = 2$, we obtain the best results: all instances can be solved to proven optimality with less computational effort (with the only exception of instances with $n = 90$).

In Table 7, we compare DSATUR [27] and DSATUR $_{\chi_{GA}}$ with $\phi_{2,1}$, i.e., the best version of DSATUR $_{\chi_{GA}}$.

Thanks to the new version of DSATUR we are now able to reduce the computing time for all instances. The effect of the strategy is to reduce the number of computed bounds and, at the same time, to reduce the number of useless bounds, i.e., bounds that cannot prune the node.

6. CONCLUSION

In this article, we have considered a series of different lower bounding techniques for DSATUR. We focus our

computational study on random VCP instances introduced in [27] since for hard DIMACS instances the Branch-and-Price algorithms remain the best approach overall. The principal idea is to exploit lower bounds for the VCP to prune the implicit enumeration scheme. In the literature, all the efforts have been made in the direction of a better selection of the branching node. In this article, we have shown instead the potential of exploiting fast but strong lower bounds within the branching scheme. These bounds can potentially lead to a significant reduction of time and nodes for DSATUR for instances with high density.

For medium and low density graphs, the bounding techniques are only effective in reducing the number of explored nodes but not the computational effort. To extend the effectiveness of the bounding techniques to these classes of instances, an interesting stream of research may concern the relation between the branching strategy and the bounding techniques. The objective would be to derive new node selection rules which can further improve the performance of DSATUR. A second promising stream of research can be the exploitation of the bound χ_f^* as a bounding procedure within DSATUR for low density graphs.

ACKNOWLEDGMENTS

The authors want to thank Denis Cornaz and Enrico Malaguti for stimulating discussions on the topic and for their contributions. The authors thank three anonymous referees for their careful reading and insightful comments.

REFERENCES

- [1] N. Barnier and P. Brisset, Graph coloring for air traffic flow management, *Ann Oper Res* 130 (2004), 163–178.
- [2] C. Bentz, M.C. Costa, D. de Werra, C. Picouleau, and B. Ries, On a graph coloring problem arising from discrete tomography, *Networks* 51 (2008), 256–267.
- [3] D. Bréaz, New methods to color the vertices of a graph, *Commun ACM* 22 (1979), 251–256.
- [4] M. Caramia and P. Dell’Olmo, Solving the minimum-weighted coloring problem, *Networks* 38 (2001), 88–101.
- [5] F. Chow and J. Hennessy, The priority-based coloring approach to register allocation, *ACM Trans Program Lang Syst* 12 (1990), 501–536.
- [6] W. Cook, S. Held, and E. Sewell, Maximum-weight stable sets and safe lower bounds for graph coloring, *Math Program Comput* 4 (2012), 363–381.
- [7] D. Cornaz, F. Furini, and E. Malaguti, Solving vertex coloring problems as maximum weight stable set problems, *Discr Appl Math*, 2016 (to appear).
- [8] D. Cornaz and V. Jost, A one-to-one correspondence between colorings and stable sets, *Oper Res Lett* 36 (2008), 673–676.
- [9] S. Das, G. Ghidini, A. Navarra, and C. Pinotti, Localization and scheduling protocols for actor-centric sensor networks, *Networks* 59 (2012), 299–319.
- [10] F. Furini, V. Gabrel, and I.-C. Ternier, DSATUR code, Web-page, Available at: www.lamsade.dauphine.fr/coloring, last accessed October, 2016.
- [11] F. Furini, V. Gabrel, and I.-C. Ternier, Lower bounding techniques for DSATUR-based branch and bound, *Electron Notes Discr Math* 52 (2016), 149–156.
- [12] M. Gamache, A. Hertz, and J. O. Ouellet, A graph coloring model for a feasibility problem in monthly crew scheduling with preferential bidding, *Comp Oper Res* 34 (2007), 2384–2395.
- [13] M. Garey and D. Johnson, *Computers and intractability: A guide to the theory of NP-completeness*, W. H. Freeman & Co, New York, 1979.
- [14] M. Grötschel, L. Lovász, and A. Schrijver, The ellipsoid method and its consequences in combinatorial optimization, *Combinatorica* 1 (1981), 169–197.
- [15] S. Gualandi and F. Malucelli, Exact solution of graph coloring problems via constraint programming and column generation, *INFORMS J Comput* 24 (2012), 81–100.
- [16] J. Hao and Q. Wu, A review on algorithms for maximum clique problems, *Eur J Oper Res* 242 (2015), 693–709.
- [17] A. Hoffman and L. Howes, On eigenvalues and colorings of graphs, *Ann NY Acad Sci* 175 (1970), 238–242.
- [18] F. Leighton, A graph coloring algorithm for large scheduling problems, *J Res Nat Bur Stand* 84 (1979), 489–506.
- [19] E. Malaguti, M. Monaci, and P. Toth, An exact approach for the vertex coloring problem, *Discr Optim* 8 (2011), 174–190.
- [20] E. Malaguti and P. Toth, A survey on vertex coloring problems, *Int Trans Oper Res* 17 (2010), 1–34.
- [21] A. Mehrotra and M. Trick, A column generation approach for graph coloring, *INFORMS J Comput* 8 (1996), 344–354.
- [22] I. Méndez-Díaz, G. Nasini, and D. Severin, A DSATUR-based algorithm for the equitable coloring problem, *Comp Oper Res* 57 (2015), 41–50.
- [23] D. Morrison, J. Sauppe, E. Sewell, and S. Jacobson, A wide branching strategy for the graph coloring problem, *INFORMS J Comput* 26 (2014), 704–717.
- [24] R. Opsut and F. Roberts, *I*-Colorings, *I*-Phasings, and *I*-Intersection assignments for graphs, and their applications, *Networks* 13 (2007), 327–345.
- [25] P.R.J. Ostergård, A fast algorithm for the maximum clique problem, *Discr Appl Math* 120 (2002), 197–207.
- [26] G. Palubeckis, On the recursive largest first algorithm for graph colouring, *Int J Comp Math* 85 (2008), 191–200.
- [27] P. San Segundo, A new DSATUR-based algorithm for exact vertex coloring, *Comp Oper Res* 39 (2012), 1724–1733.
- [28] P. San Segundo, D. Rodríguez-Losada, and A. Jimenez, An exact bit-parallel algorithm for the maximum clique problem, *Comp Oper Res* 38 (2011), 571–581.
- [29] A. Schrijver, *Combinatorial optimization*, Springer-Verlag, Berlin, Heidelberg, 2002.
- [30] E. Sewell, “An improved algorithm for exact graph coloring,” *Cliques, coloring, and satisfiability*, D.S. Johnson and M.A. Trick (Editors), American Mathematical Society, Providence, Rhodes Island, USA, 1996, pp. 359–373.
- [31] N. Zufferey, P. Amstutz, and P. Giaccari, Graph colouring approaches for a satellite range scheduling problem, *J Scheduling* 11 (2008), 263–277.

APPENDIX

As discussed in Section 2, Tables 8 and 9 report the results of DSATUR [27], our reimplementation of DSATUR [10], MMT-BP [19] and Exactcolors [6] for DIMACS instances. Following the conclusions of [27], for hard DIMACS instances, the Branch-and-Price algorithm remains the best approach overall. However, DSATUR is competitive for a subset of instances.

Tables 10 and 11 report the results of the lower bounds presented in Section 3 on DIMACS instances.

The number of vertices in G_A can be huge; for these instances the graph G_A cannot be built and accordingly the bound χ_{G_A} cannot be obtained. This is the exhaustive list of these instances: 1-Insertions_6, 2-FullIns_5, 2-Insertions_5, 3-FullIns_4, 3-FullIns_5, 3-Insertions_5, 4-FullIns_4, 4-FullIns_5, 4-Insertions_4, 5-FullIns_4, abb313GPIA, ash331GPIA, ash608GPIA, ash958GPIA, C2000.5, C4000.5, DSJC1000.1, DSJC1000.5, DSJC1000.9, DSJC500.1,

DSJC500.5, DSJR500.1, DSJR500.5, flat1000_50_0, flat1000_60_0, flat1000_76_0, fpsol2.i.1, fpsol2.i.2, fpsol2.i.3, inithx.i.1, inithx.i.2, inithx.i.3, latin_square, le450_15a, le450_15b, le450_15c, le450_15d, le450_25a, le450_25b, le450_25c, le450_5a, le450_5b, le450_5c, le450_5d, qg.order100, qg.order30, qg.order40, qg.order60, school1, school1_nsh, wap01a, wap02a, wap03a, wap04a, wap05a, wap06a, wap07a, wap08a, will199GPIA. For these instances we report “-”. For the cases where the time limit is reached, we report the χ_{G_A} bound computed with $\bar{\alpha}(G_A)$ equal to the upper bound returned by CPLEX at the time limit. For these cases, since the quality of $\bar{\alpha}(G_A)$ is really poor, the bound χ_{G_A} can be negative.

Table 12 reports the results of DSATUR_ω which is DSATUR with bound ω and $\phi_{u,g} = \text{true}$. The bound ω allows reducing the number of nodes (not as substantially as χ_{G_A}). Unfortunately, this version is not faster than DSATUR and it is not able to solve instances starting from density $d = 0.3$.

TABLE 8. Exact VCP algorithms on DIMACS instances.

Instance			DSATUR						Branch-and-price algorithms							
			[27]		Reimplementation [10]				MMT-BP [19]			Exactcolors [6]				
Instance	<i>n</i>	<i>m</i>	<i>d</i>	<i>UB</i>	Time	<i>UB</i>	Time	Nodes	<i>LB</i>	<i>UB</i>	Time	<i>LB</i>	<i>UB</i>	Time	Nodes	
1-FullIns_4	93	593	0.1	5	0.2	5	0.0	3,013	4	5	tl	4	5	tl	39,851	
1-FullIns_5	282	3,247	0.1	6	tl	6	0.0	4,611	4	6	tl	4	6	tl	2,893	
1-Insertions_4	67	232	0.1	5	240.0	5	tl	3,285,368,292	3	5	tl	4	5	tl	48,673	
1-Insertions_5	202	1,227	0.1	6	tl	6	tl	1,941,304,976	3	6	tl	4	6	tl	535	
1-Insertions_6	607	6,337	≈0.0	7	tl	7	tl	984,504,060	3	7	tl	—	—	tl	—	
2-FullIns_3	52	201	0.2	5	0.0	5	0.0	6	5	5	2.9	5	5	0.0	1	
2-FullIns_4	212	1,621	0.1	6	tl	6	0.0	69	5	6	tl	5	6	tl	3,057	
2-FullIns_5	852	12,201	≈0.0	7	tl	7	0.0	40,482	5	7	tl	5	7	tl	289	
2-Insertions_4	149	541	0.0	5	tl	5	tl	2,282,050,125	3	5	tl	3	5	tl	2,149	
2-Insertions_5	597	3,936	0.0	6	tl	6	tl	1,028,588,270	3	6	tl	3	6	tl	25	
3-FullIns_3	80	346	0.1	6	tl	6	0.0	7	6	6	2.9	6	6	0.0	1	
3-FullIns_4	405	3,524	≈0.0	7	tl	7	0.0	129	6	7	tl	6	7	tl	989	
3-FullIns_5	2,030	33,751	≈0.0	8	tl	8	tl	665,789,658	6	8	tl	6	8	tl	115	
3-Insertions_3	56	110	0.1	4	0.6	4	1.5	3,434,261	3	4	tl	3	4	tl	85,789	
3-Insertions_4	281	1,046	0.0	5	tl	5	tl	1,608,405,161	3	5	tl	3	5	tl	211	
3-Insertions_5	1,406	9,695	0.0	6	tl	6	tl	497,689,431	2	6	tl	—	—	tl	—	
4-FullIns_3	114	541	0.1	7	tl	7	0.0	8	7	7	3.4	7	7	0.0	1	
4-FullIns_4	690	6,650	≈0.0	8	tl	8	0.0	128	7	8	tl	7	8	tl	1,137	
4-FullIns_5	4,146	77,305	≈0.0	9	tl	9	tl	423,272,354	7	9	tl	7	9	tl	33	
4-Insertions_3	79	156	0.1	4	351.0	4	tl	3,229,375,685	3	4	tl	3	4	tl	32,149	
4-Insertions_4	475	1,795	≈0.0	5	tl	5	tl	1,250,788,090	3	5	tl	3	5	tl	31	
5-FullIns_3	154	792	0.1	8	tl	8	0.0	9	8	8	4.6	8	8	0.0	1	
5-FullIns_4	1,085	11,395	0.0	9	tl	9	0.0	153	8	9	tl	8	9	tl	737	
abb313GPIA	1,557	53,356	≈0.0	10	tl	10	tl	785,420,931	8	9	tl	—	—	tl	—	
anna	138	493	0.1	11	≈0.0	11	0.0	1	11	11	3.6	11	11	0.0	1	
ash331GPIA	662	4,181	≈0.0	4	0.0	4	0.0	1,131	4	4	45.9	4	6	tl	45	
ash608GPIA	1,216	7,844	0.0	4	0.1	4	0.0	2,427	4	4	tl	—	—	tl	—	
ash958GPIA	1,916	12,506	0.0	4	0.4	5	tl	234,359,946	3	4	tl	—	—	tl	—	
C2000.5	2,000	999,836	0.5	—	—	216	tl	45475007	—	—	—	—	—	tl	—	
C4000.5	4,000	4,000,268	0.5	—	—	—	tl	—	—	—	—	—	—	tl	—	
david	87	406	0.1	11	0.0	11	0.0	1	11	11	0.2	11	11	0.0	1	
DSJC1000.1	1,000	49,629	0.1	—	—	25	tl	311,540,866	5	20	tl	—	—	tl	—	
DSJC1000.5	1,000	249,826	0.5	—	—	114	tl	144,353,376	13	84	tl	—	—	tl	—	
DSJC1000.9	1,000	449,449	0.9	—	—	—	tl	—	51	224	tl	—	—	tl	—	
DSJC125.1	125	736	0.1	5	0.0	5	0.0	290	5	5	42.0	5	6	tl	2,097	
DSJC125.5	125	3,891	0.5	19	tl	19	tl	1,007,429,681	17	17	tl	17	23	tl	4,487	
DSJC125.9	125	6,961	0.9	46	tl	45	tl	701,674,554	44	44	tl	44	50	tl	32,661	
DSJC250.1	250	3,218	0.1	9	tl	9	tl	1,017,076,072	6	8	tl	—	—	tl	2,152	
DSJC250.5	250	15,668	0.5	34	tl	35	tl	569,582,279	20	28	tl	26	37	tl	535	
DSJC250.9	250	27,897	0.9	82	tl	—	tl	—	71	72	tl	71	90	tl	2,647	
DSJC500.1	500	12,458	0.1	14	tl	14	tl	603,420,521	5	12	tl	—	—	tl	—	
DSJC500.5	500	62,624	0.5	62	tl	62	tl	321,102,532	16	48	tl	—	—	tl	—	
DSJC500.9	500	112,437	0.9	—	—	164	tl	178,267,456	123	127	tl	123	163	tl	287	
DSJR500.1c	500	121,275	≈1.0	85	tl	—	tl	—	85	85	288.5	85	88	tl	1,421	
DSJR500.1	500	3,555	≈0.0	12	0.0	12	0.1	117,132	12	12	35.3	12	14	tl	2,321	
DSJR500.5	500	58,862	0.5	130	tl	134	tl	1,026,643,225	122	122	342.2	122	132	tl	139	
flat1000_50_0	1,000	245,000	0.5	—	—	112	tl	161,099,104	—	—	—	—	—	tl	tl	
flat1000_60_0	1,000	245,830	0.5	—	—	112	tl	142,816,890	—	—	—	—	—	tl	tl	
flat1000_76_0	1,000	246,708	0.5	—	—	111	tl	125,707,629	—	—	—	—	—	tl	tl	
fpsol2.i.1	496	11,654	0.1	65	0.0	65	0.0	1	65	65	10.6	65	65	0.5	1	
fpsol2.i.2	451	8,691	0.1	30	0.0	30	0.0	1	30	30	11.2	30	30	0.3	1	
fpsol2.i.3	425	8,688	0.1	30	0.0	30	0.0	1	30	30	10.0	30	30	0.4	1	
games120	120	638	0.1	9	0.0	9	0.0	1	9	9	0.2	9	9	0.0	1	
huck	74	301	0.1	11	0.0	11	0.0	1	11	11	0.2	11	11	0.0	1	
inithx.i.1	864	18,707	0.1	54	0.0	54	0.0	1	54	54	21.0	54	54	1.6	1	
inithx.i.2	645	13,979	0.1	31	0.0	31	0.0	1	31	31	9.2	31	31	1.4	1	
inithx.i.3	621	13,969	0.1	31	0.0	31	0.0	1	31	31	9.9	31	31	0.5	1	
jean	80	254	0.1	10	0.0	10	0.0	1	10	10	0.2	10	10	0.0	1	

TABLE 9. Exact VCP algorithms on DIMACS instances.

Instance			DSATUR						Branch-and-price algorithms							
			[27]		Reimplementation [10]			MMT-BP [19]			Exactcolors [6]					
Instance	n	m	d	UB	Time	UB	Time	Nodes	LB	UB	Time	LB	UB	Time	Nodes	
latin_square	900	307,350	0.8	130	tl	153	tl	101214127	90	102	tl	90	129	tl	53	
le450_15a	450	8,168	0.1	16	tl	17	tl	2,067,792,547	15	15	0.4	15	17	tl	229	
le450_15b	450	8,169	0.1	16	tl	16	tl	972,319,400	15	15	0.2	15	17	tl	241	
le450_15c	450	16,680	0.2	22	tl	22	tl	639,434,972	15	15	3.1	—	—	tl	tl	
le450_15d	450	16,750	0.2	23	tl	23	tl	635,710,737	15	15	3.8	—	—	tl	tl	
le450_25a	450	8,260	0.1	25	0.0	25	0.0	1	25	25	0.1	25	25	2.6	1	
le450_25b	450	8,263	0.1	25	0.0	25	0.0	1	25	25	0.1	25	25	2.8	1	
le450_25c	450	17,343	0.2	—	—	27	tl	677,939,432	25	25	1,356.6	25	28	tl	187	
le450_5a	450	5,714	0.1	—	—	9	tl	727,964,669	5	5	0.3	—	—	tl	tl	
le450_5b	450	5,734	0.1	—	—	9	tl	747,642,318	5	5	0.2	—	—	tl	tl	
le450_5c	450	9,803	0.1	5	0.0	5	0.0	2,892	5	5	0.1	—	—	tl	tl	
le450_5d	450	9,757	0.1	5	98.1	5	51.6	20,397,241	5	5	0.2	—	—	tl	tl	
miles1000	128	3,216	0.4	42	0.0	42	0.0	45	42	42	0.2	42	42	0.1	1	
miles1500	128	5,198	0.6	73	0.0	73	0.0	1	7	73	0.1	73	73	0.2	1	
miles250	128	387	≈0.0	8	0.0	8	0.0	1	8	8	5	8	8	0.0	1	
miles500	128	1,170	0.1	20	0.0	20	0.0	1	20	20	3.7	20	20	0.0	1	
miles750	128	2,113	0.3	31	0.0	31	0.0	44	31	31	0.2	31	31	0.1	1	
mug100_1	100	166	≈0.0	4	0.0	4	tl	5,348,610,817	4	4	14.4	4	4	1.5	1	
mug100_25	100	166	≈0.0	4	0.0	4	tl	5,230,617,855	4	4	12	4	4	1.4	1	
mug88_1	88	146	≈0.0	4	324.0	4	460.7	1,694,387,669	4	4	9.6	—	—	tl	tl	
mug88_25	88	146	≈0.0	4	191.0	4	290.6	1,062,827,981	4	4	10.6	—	—	tl	tl	
mulsol.i.1	197	3,925	0.2	49	0.0	49	0.0	1	49	49	0.2	49	49	0.1	1	
mulsol.i.2	188	3,885	0.2	31	0.0	31	0.0	1	31	31	4.7	31	31	0.1	1	
mulsol.i.3	184	3,916	0.2	31	0.0	31	0.0	1	31	31	0.2	31	31	0.0	1	
mulsol.i.4	185	3,946	0.2	31	0.0	31	0.0	1	31	31	0.2	31	31	0.0	1	
mulsol.i.5	186	3,973	0.2	31	0.0	31	0.0	1	31	31	6	31	31	0.0	1	
myciel6	95	755	0.2	7	tl	7	tl	2,106,152,516	4	7	tl	5	7	tl	24,537	
myciel7	191	2,360	0.1	8	tl	8	tl	1,573,252,182	5	8	tl	5	8	tl	4,371	
qg.order30	900	26,100	0.1	30	0.0	30	0.0	2,491	30	30	0.2	—	—	tl	tl	
qg.order40	1,600	62,400	≈0.0	40	0.2	40	0.2	58,971	40	40	2.9	—	—	tl	tl	
qg.order60	3,600	212,400	≈0.0	61	tl	62	tl	377,150,626	60	60	3.8	—	—	tl	tl	
queen10_10	100	1,470	0.3	12	466.0	11	tl	1,676,596,671	11	11	686.9	11	14	tl	3,777	
queen11_11	121	1,980	0.3	13	tl	13	tl	1,538,642,053	11	11	tl	11	14	tl	1,675	
queen12_12	144	2,596	0.3	14	tl	14	tl	1,430,010,102	12	13	tl	12	16	tl	573	
queen13_13	169	3,328	0.2	15	tl	15	tl	1,340,956,423	13	14	tl	13	17	tl	335	
queen14_14	196	4,186	0.2	16	tl	16	tl	1,187,949,544	14	15	tl	14	19	tl	91	
queen15_15	225	5,180	0.2	18	tl	18	tl	1,167,846,294	15	16	tl	—	—	tl	tl	
queen16_16	256	6,320	0.2	19	tl	19	tl	1,047,030,161	16	17	tl	16	21	tl	115	
queen8_12	96	1,368	0.3	12	0.0	12	0.0	375	12	12	0.2	12	12	11.3	261	
queen8_8	64	728	0.4	9	0.0	9	0.5	668,699	9	9	3.6	9	13	tl	30,057	
queen9_9	81	1,056	0.3	10	3.0	10	244.5	300,489,064	10	10	36.6	10	12	tl	12,653	
r1000.1c	1,000	485,090	≈1.0	—	—	—	—	—	—	—	—	—	—	—	—	
school1	385	19,095	0.3	14	0.1	14	4.5	383	14	14	0.4	14	14	1,290.1	777	
school1_nsh	352	14,612	0.2	14	0.4	14	1.7	1,001	14	14	17	14	14	1,060.9	697	
wap01a	2,368	110,871	≈0.0	46	tl	51	tl	327,214,469	40	43	tl	—	—	tl	tl	
wap02a	2,464	111,742	≈0.0	46	tl	45	tl	507,088,624	40	42	tl	—	—	tl	tl	
wap03a	4,730	286,722	≈0.0	—	—	53	tl	353,433,910	40	47	tl	—	—	tl	tl	
wap04a	5,231	294,902	≈0.0	—	—	48	tl	215,930,024	40	44	tl	—	—	tl	tl	
wap05a	905	43,081	0.1	50	0.0	50	0.0	1	50	50	293.2	50	50	6.0	1	
wap06a	947	43,571	0.1	47	tl	45	tl	565,243,140	40	40	175	—	—	tl	tl	
wap07a	1,809	103,368	0.1	44	tl	45	tl	341,848,594	40	42	tl	—	—	tl	tl	
wap08a	1,870	104,176	0.1	45	tl	46	tl	365,009,521	40	42	tl	—	—	tl	tl	
will199GPIA	701	6,772	≈0.0	7	tl	7	0.0	693	7	7	80.7	7	7	3.8	1	
zeroin.i.1	211	4,100	0.2	49	0.0	49	0.0	1	49	49	3.8	49	49	0.1	1	
zeroin.i.2	211	3,541	0.2	30	0.0	30	0.0	1	30	30	4.4	30	30	0.0	1	
zeroin.i.3	206	3,540	0.2	30	0.0	30	0.0	1	30	30	4.5	30	30	0.0	1	

TABLE 10. Comparison between Lower Bounds on $\chi(G)$ for DIMACS instances.

Instance				Value						Time					
Name	n	m	d	χ_f^*	χ_{GA}	ω	χ_α	χ_H^*	ω^h	χ_f^*	χ_{GA}	ω	χ_α	χ_H^*	ω^h
1-FullIns_4	93	593	0.1	4	4	3	2	2	3	0.0	21.5	0.0	0.0	0.0	0.0
1-FullIns_5	282	3,247	0.1	4	−36,091	3	2	2	3	0.2	tl	0.0	0.0	0.0	0.0
1-Insertions_4	67	232	0.1	3	3	2	2	2	2	0.1	4.9	0.0	0.0	0.0	0.0
1-Insertions_5	202	1,227	0.1	3	3	2	2	2	2	6.0	tl	0.0	0.0	0.0	0.0
1-Insertions_6	607	6,337	≈0.0	4	−	2	2	2	2	1,427.6	tl	0.0	0.1	0.1	0.0
2-FullIns_3	52	201	0.2	5	4	4	2	2	4	0.0	0.5	0.0	0.0	0.0	0.0
2-FullIns_4	212	1,621	0.1	5	−20,532	4	2	2	4	0.1	tl	0.0	0.0	0.0	0.0
2-FullIns_5	852	12,201	≈0.0	5	−	4	2	2	4	1.9	tl	0.0	0.0	0.2	0.0
2-Insertions_4	149	541	≈0.0	3	3	2	2	2	2	2.1	520.3	0.0	0.0	0.0	0.0
2-Insertions_5	597	3,936	≈0.0	3	−	2	2	2	2	381.6	tl	0.0	0.0	0.1	0.0
3-FullIns_3	80	346	0.1	6	5	5	2	2	5	0.0	2.9	0.0	0.0	0.0	0.0
3-FullIns_4	405	3,524	≈0.0	6	−	5	2	2	5	0.1	tl	0.0	0.5	0.0	0.0
3-FullIns_5	2,030	33,751	≈0.0	6	−	5	−	2	5	7.0	tl	0.0	tl	2.3	0.0
3-Insertions_3	56	110	0.1	3	3	2	2	2	2	0.1	1.8	0.0	0.0	0.0	0.0
3-Insertions_4	281	1,046	≈0.0	3	−38,012	2	2	2	2	25.3	tl	0.0	0.0	0.0	0.0
3-Insertions_5	1,406	9,695	≈0.0	−	−	2	2	2	2	tl	tl	0.0	4.9	0.7	0.0
4-FullIns_3	114	541	0.1	7	6	6	2	2	6	0.0	15.8	0.0	0.0	0.0	0.0
4-FullIns_4	690	6,650	≈0.0	7	−	6	2	2	6	0.2	tl	0.0	0.3	0.1	0.0
4-FullIns_5	4,146	77,305	≈0.0	7	−	6	−	2	6	17.5	tl	0.1	tl	21.1	0.0
4-Insertions_3	79	156	0.1	3	3	2	2	2	2	0.2	5.4	0.0	0.0	0.0	0.0
4-Insertions_4	475	1,795	≈0.0	3	−	2	2	2	2	193.7	tl	0.0	0.0	0.0	0.0
5-FullIns_3	154	792	0.1	8	7	7	2	2	7	0.0	45.6	0.0	0.0	0.0	0.0
5-FullIns_4	1,085	11,395	≈0.0	8	−	7	−	2	7	0.5	tl	0.0	tl	0.3	0.0
abb313GPIA	1,557	53,356	≈0.0	−	−	8	4	3	2	tl	tl	0.0	0.1	1.0	0.0
anna	138	493	0.1	11	11	11	1	3	6	0.0	4.7	0.0	0.0	0.0	0.0
ash331GPIA	662	4,181	≈0.0	4	−	3	−	2	2	6.0	tl	0.0	tl	0.1	0.0
ash608GPIA	1,216	7,844	≈0.0	−	−	3	−	2	2	tl	tl	0.0	tl	0.4	0.0
ash958GPIA	1,916	12,506	≈0.0	−	−	3	−	2	2	tl	tl	0.0	tl	1.9	0.0
C2000.5	2,000	999,836	0.5	−	−	−	−	23	2	tl	tl	tl	tl	2.4	0.0
C4000.5	4,000	4,000,268	0.5	−	−	−	−	32	2	tl	tl	tl	tl	19.8	0.0
david	87	406	0.1	11	11	11	2	3	7	0.0	0.7	0.0	0.0	0.0	0.0
DSJC1000.1	1,000	49,629	0.1	−	−	6	−	6	2	tl	tl	0.0	tl	0.3	0.0
DSJC1000.5	1,000	249,826	0.5	−	−	15	66	17	2	tl	tl	497.6	609.9	0.3	0.0
DSJC1000.9	1,000	449,449	0.9	−	−	−	166	48	4	tl	tl	tl	0.0	0.3	0.0
DSJC125.1	125	736	0.1	5	4	4	3	3	2	1.7	1,125.61	0.0	10.9	0.0	0.0
DSJC125.5	125	3,891	0.5	16	14	10	12	6	2	0.6	108.2	0.0	0.0	0.0	0.0
DSJC125.9	125	6,961	0.9	43	43	34	31	16	10	0.2	0.2	10.2	0.0	0.0	0.0
DSJC250.1	250	3,218	0.1	7	−27,656	4	−	3	2	451.2	tl	0.0	tl	0.0	0.0
DSJC250.5	250	15,668	0.5	26	−15,206	12	20	8	2	10.0	tl	0.0	0.0	0.0	0.0
DSJC250.9	250	27,897	0.9	71	71	−	50	23	7	3.6	5.6	tl	0.0	0.0	0.0
DSJC500.1	500	12,458	0.1	−	−	5	−	4	2	tl	tl	0.0	tl	0.0	0.0
DSJC500.5	500	62,624	0.5	43	−	13	38	12	3	285.5	tl	4.7	4.6	0.0	0.0
DSJC500.9	500	112,437	0.9	123	122	−	100	33	11	50.1	1,060.79	−	0	0.06	0
DSJR500.1c	500	121,275	≈1.0	85	85	12	38	25	2	10.3	2.0	0.0	0.0	0.1	0.0
DSJR500.1	500	3,555	≈0.0	12	−	−	−	4	2	0.8	tl	tl	tl	0.0	0.0
DSJR500.5	500	58,862	0.5	122	−	122	71	8	72	39.5	tl	1,022.8	0.0	0.0	0.0
flat1000_50_0	1,000	245,000	0.5	−	−	15	50	16	2	tl	tl	290.7	20.6	0.3	0.0
flat1000_60_0	1,000	245,830	0.5	−	−	15	58	16	2	tl	tl	339.9	119.8	0.3	0.0
flat1000_76_0	1,000	246,708	0.5	−	−	15	66	16	2	tl	tl	703.3	559.0	0.3	0.0
fpsol2.i.1	496	11,654	0.1	65	−	65	1	3	2	0.4	tl	0.0	0.0	0.0	0.0
fpsol2.i.2	451	8,691	0.1	30	−	30	1	2	2	0.3	tl	0.0	0.0	0.0	0.0
fpsol2.i.3	425	8,688	0.1	30	−	30	1	2	2	0.4	tl	0.0	0.0	0.0	0.0
games120	120	638	0.1	9	9	9	5	3	2	0.0	5.0	0.0	0.0	0.0	0.0
huck	74	301	0.1	11	11	11	2	3	4	0.0	0.3	0.0	0.0	0.0	0.0
inithx.i.1	864	18,707	0.1	54	−	54	1	2	2	1.5	tl	0.0	0.1	0.1	0.0
inithx.i.2	645	13,979	0.1	31	−	31	1	2	2	1.3	tl	0.0	0.0	0.1	0.0
inithx.i.3	621	13,969	0.1	31	−	31	1	2	2	0.4	tl	0.0	0.0	0.1	0.0
jean	80	254	0.1	10	10	10	2	3	3	0.0	0.5	0.0	0.0	0.0	0.0
latin_square	900	307,350	0.8	90	−	−	90	17	90	48.7	tl	tl	0.0	0.2	0.0

TABLE 11. Comparison between lower bounds on $\chi(G)$ for DIMACS instances.

Instance				Value						Time					
Name	n	m	d	χ_f^*	χ_{GA}	ω	χ_α	χ_H^*	ω^h	χ_f^*	χ_{GA}	ω	χ_α	χ_H^*	ω^h
le450_15a	450	8,168	0.1	15	–	15	–	5	2	41.8	tl	0.0	tl	0.0	0.0
le450_15b	450	8,169	0.1	15	–	15	–	6	3	38.3	tl	0.0	tl	0.0	0.0
le450_15c	450	16,680	0.2	15	–	15	–	7	4	621.2	tl	0.0	tl	0.0	0.0
le450_15d	450	16,750	0.2	15	–	15	–	6	2	417.8	tl	0.0	tl	0.0	0.0
le450_25a	450	8,260	0.1	25	–	25	–	8	3	2.4	tl	0.0	tl	0.0	0.0
le450_25b	450	8,263	0.1	25	–	25	–	8	2	2.5	tl	0.0	tl	0.0	0.0
le450_25c	450	17,343	0.2	25	–	25	–	9	2	42.2	tl	0.0	tl	0.0	0.0
le450_5a	450	5,714	0.1	5	–	5	–	3	2	1,415.6	tl	0.0	tl	0.0	0.0
le450_5b	450	5,734	0.1	–	–	5	–	3	2	tl	tl	0.0	tl	0.0	0.0
le450_5c	450	9,803	0.1	5	–	5	–	3	2	562.0	tl	0.0	tl	0.0	0.0
le450_5d	450	9,757	0.1	5	–	5	–	3	2	737.6	tl	0.0	tl	0.0	0.0
miles1000	128	3,216	0.4	42	42	42	16	7	29	0.1	1.0	0.0	0.0	0.0	0.0
miles1500	128	5,198	0.6	73	71	73	25	8	51	0.1	0.4	0.0	0.0	0.0	0.0
miles250	128	387	≈ 0.0	8	8	8	2	4	3	0.0	3.7	0.0	0.0	0.0	0.0
miles500	128	1,170	0.1	20	19	20	7	5	12	0.0	6.6	0.0	0.0	0.0	0.0
miles750	128	2,113	0.3	31	31	31	10	6	11	0.0	2.9	0.0	0.0	0.0	0.0
mug100_1	100	166	≈ 0.0	4	3	3	3	2	2	1.6	2.1	0.0	11.1	0.0	0.0
mug100_25	100	166	≈ 0.0	4	3	3	3	2	2	1.2	1.2	0.0	25.8	0.0	0.0
mug88_1	88	146	≈ 0.0	4	3	3	3	2	2	0.6	1.9	0.0	0.0	0.0	0.0
mug88_25	88	146	≈ 0.0	4	3	3	3	2	2	0.8	1.7	0.0	0.0	0.0	0.0
multsol.i.1	197	3,925	0.2	49	49	49	1	4	2	0.1	8.1	0.0	0.0	0.0	0.0
multsol.i.2	188	3,885	0.2	31	31	31	2	2	2	0.1	7.3	0.0	0.0	0.0	0.0
multsol.i.3	184	3,916	0.2	31	31	31	2	2	2	0.0	4.9	0.0	0.0	0.0	0.0
multsol.i.4	185	3,946	0.2	31	31	31	2	2	2	0.0	4.9	0.0	0.0	0.0	0.0
multsol.i.5	186	3,973	0.2	31	31	31	2	2	2	0.0	6.8	0.0	0.0	0.0	0.0
myciel6	95	755	0.2	4	4	2	2	2	2	0.1	18.5	0.0	0.0	0.0	0.0
myciel7	191	2,360	0.1	5	4	2	2	2	2	1.1	1,454.3	0.0	0.0	0.0	0.0
qg.order30	900	26,100	0.1	30	–	30	30	30	30	182.0	tl	0.0	0.0	0.2	0.0
qg.order40	1,600	62,400	≈ 0.0	–	–	40	40	39	40	tl	tl	0.0	0.0	1.0	0.0
qg.order60	3,600	212,400	≈ 0.0	–	–	60	60	59	60	tl	tl	0.1	0.1	14.0	0.0
queen10_10	100	1,470	0.3	10	10	10	10	8	4	0.6	30.3	0.0	0.0	0.0	0.0
queen11_11	121	1,980	0.3	11	11	11	11	9	4	0.9	95.0	0.0	0.0	0.0	0.0
queen12_12	144	2,596	0.3	12	12	12	12	10	4	1.7	99.6	0.0	0.0	0.0	0.0
queen13_13	169	3,328	0.2	13	13	13	13	10	4	3.1	1,151.5	0.0	0.0	0.0	0.0
queen14_14	196	4,186	0.2	14	14	14	14	11	4	4.9	tl	0.0	0.0	0.0	0.0
queen15_15	225	5,180	0.2	15	15	15	15	12	4	8.5	tl	0.0	0.0	0.0	0.0
queen16_16	256	6,320	0.2	16	16	16	16	13	4	14.3	tl	0.0	0.0	0.0	0.0
queen8_12	96	1,368	0.3	12	12	12	12	8	2	0.1	3.9	0.0	0.0	0.0	0.0
queen8_8	64	728	0.4	9	8	8	8	6	4	0.0	2.6	0.0	0.0	0.0	0.0
queen9_9	81	1,056	0.3	9	9	9	9	7	4	0.3	6.5	0.0	0.0	0.0	0.0
r1000.1c	1,000	485,090	≈ 1.0	96	95	–	41	25	8	1,239.8	403.1	tl	0.0	0.3	0.0
school1	385	19,095	0.3	14	–	14	–	9	6	2	304.6	17.5	0.2	0.0	0.0
school1_nsh	352	14,612	0.2	14	–	14	9	6	3	76.5	tl	7.2	0.0	0.0	0.0
wap01a	2,368	110,871	≈ 0.0	–	–	41	–	8	2	tl	tl	0.0	tl	3.8	0.0
wap02a	2,464	111,742	≈ 0.0	–	–	40	–	8	2	tl	tl	0.1	tl	4.3	0.0
wap03a	4,730	286,722	≈ 0.0	–	–	40	–	7	2	tl	tl	0.6	tl	31.2	0.0
wap04a	5,231	294,902	≈ 0.0	–	–	40	–	7	2	tl	tl	0.2	tl	42.4	0.0
wap05a	905	43,081	0.1	50	–	50	–	6	2	5.5	tl	0.0	tl	0.2	0.0
wap06a	947	43,571	0.1	40	–	40	11	6	2	618.8	tl	0.0	23.5	0.2	0.0
wap07a	1,809	103,368	0.1	–	–	40	–	7	2	tl	tl	0.0	tl	1.6	0.0
wap08a	1,870	104,176	0.1	–	–	40	–	7	2	tl	tl	0.0	tl	1.8	0.0
will199GPIA	701	6,772	≈ 0.0	7	–	6	–	2	2	3.6	tl	0.0	tl	0.1	0.0
zeroin.i.1	211	4,100	0.2	49	49	49	1	3	11	0.1	12.5	0.0	0.0	0.0	0.0
zeroin.i.2	211	3,541	0.2	30	30	30	1	2	3	0.0	13.8	0.0	0.0	0.0	0.0
zeroin.i.3	206	3,540	0.2	30	30	30	1	2	3	0.0	12.6	0.0	0.0	0.0	0.0

TABLE 12. DSATUR and DSATUR_ω for random instances with $n = 90$ and various densities.

Instance			DSATUR [27]				DSATUR _ω				
n	d	#	UB	Time	Nodes	Fails	UB	Time	Nodes	#bounds * \ #bounds	Fails
90	0.1	5	5.00	0.00	847	0	5	0.05	698	55.60 \ 697.40	0
90	0.2	5	7.00	0.18	233,241	0	7	17.72	166,437	22,783.60 \ 166,435.00	0
90	0.3	5	9.00	13.48	14,502,948	0	9.2	550.48	4,354,837	1,080,555.00 \ 6,663,805.40	2
90	0.4	5	11.20	242.67	235,166,299	1	11.4	–	–	1,603,050.40 \ 8,910,127.80	5
90	0.5	5	14.00	584.54	532,332,791	4	14.4	–	–	1,731,827.40 \ 8,221,246.40	5
90	0.6	5	17.20	–	–	5	17.8	–	–	2,019,824.40 \ 7,801,352.40	5
90	0.7	5	20.40	945.57	726,706,418	4	21.2	–	–	1,827,355.20 \ 6,139,712.60	5
90	0.8	5	25.60	–	–	5	26.4	–	–	1,662,574.40 \ 5,264,752.00	5
90	0.9	5	34.40	83.57	55,197,342	0	34.8	175.82	286,049	651,559.40 \ 2,069,449.20	3